



Database Systems

Lecture 9:

The Entity Relationship Model (Part 2)

Yumeng Song

Department of Computer Science

Aalborg University

yumengs@cs.aau.dk

Fall 2025

Learning Objectives



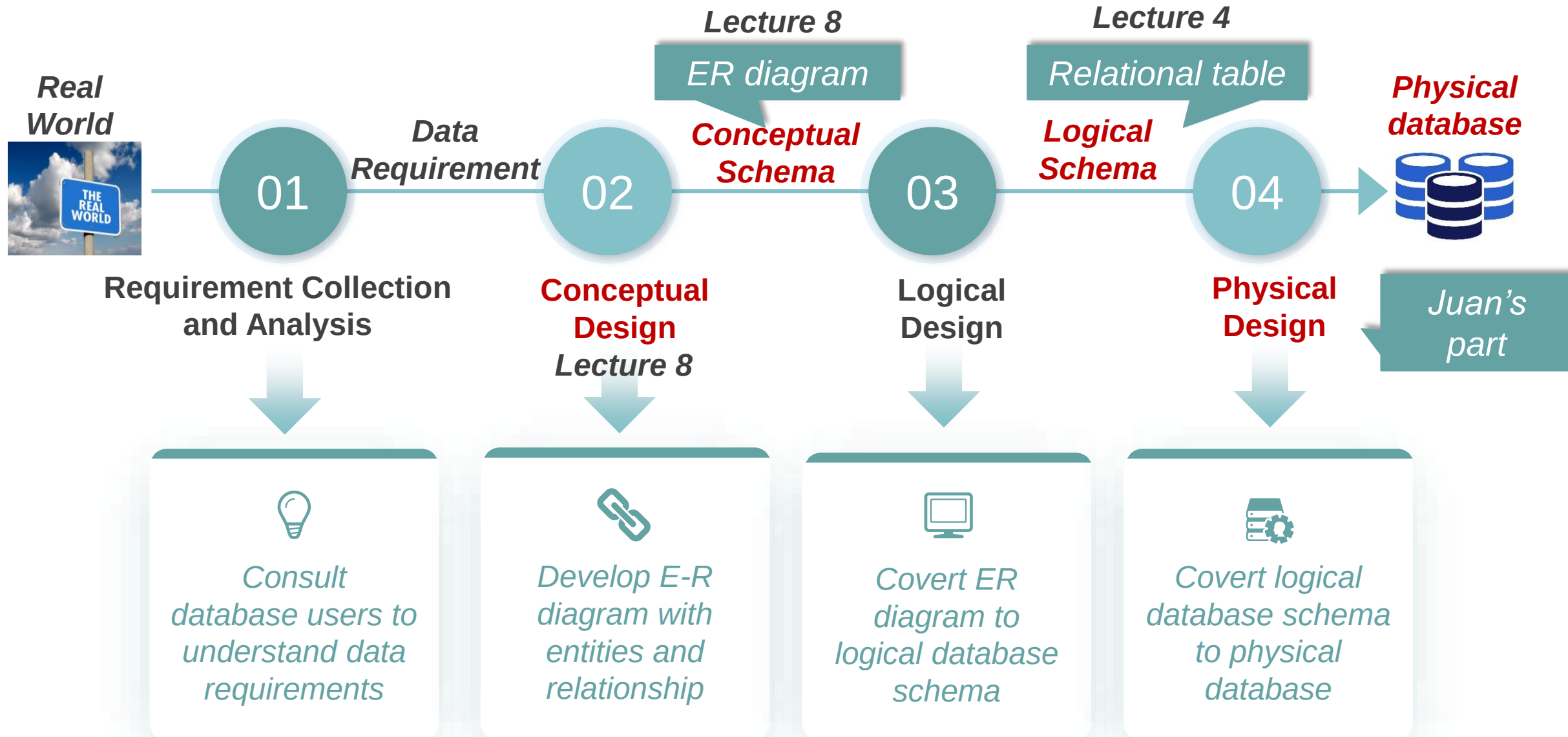
- After completing this lecture, you will be able to:
 - ✓ Create non-trivial ER diagrams
 - ✓ Use a particular ER notation properly
 - ✓ Assess the quality of an ER diagram
 - Perform and explain the mapping of ER diagrams to relations

Agenda



- Review the ER model and relational model
- Mapping basic concepts to relations
 - Entity types
 - Relationship types
- Map complex cases to relations
 - Weak entity types
 - Recursive relationship types
 - N-ary relationship types
 - Special attributes
- Relational modelling of specialization

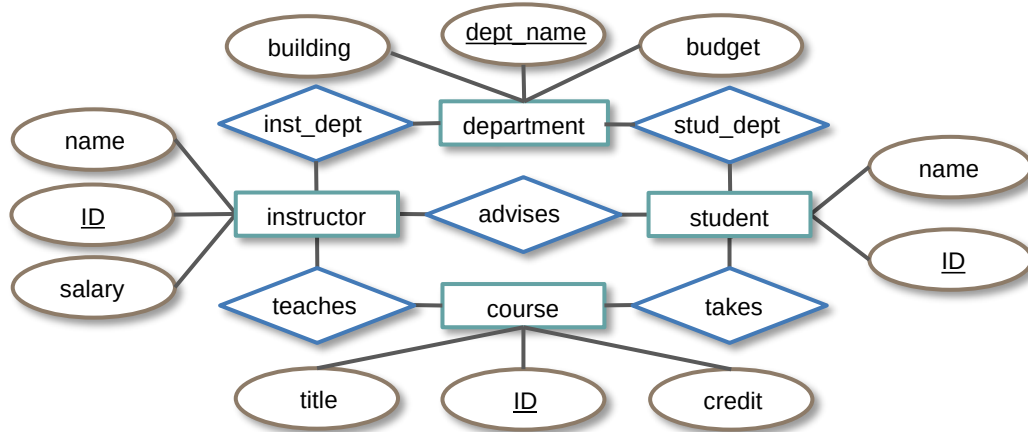
Steps of Database Design



ER Model vs. Relational Model



Conceptual Schema



ER Model

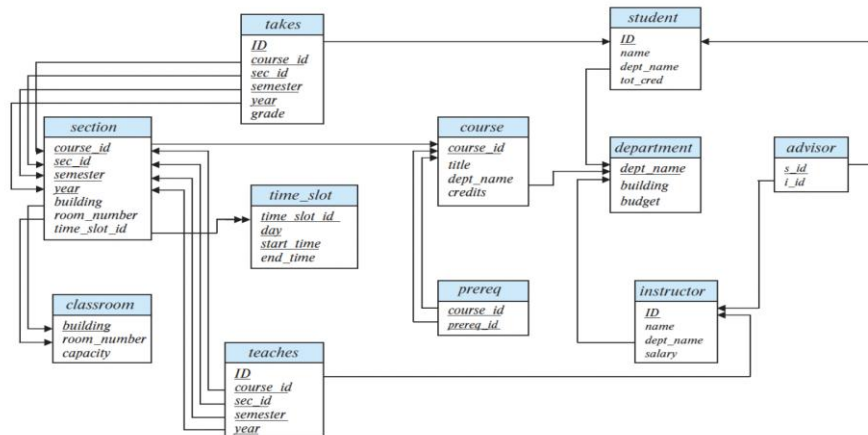
- Entities
- Relationships
- Attributes

✓ *Human-understandable*

Provides a conceptual view of the data, focusing on meaning and structure.

How to translate?

Logical Schema



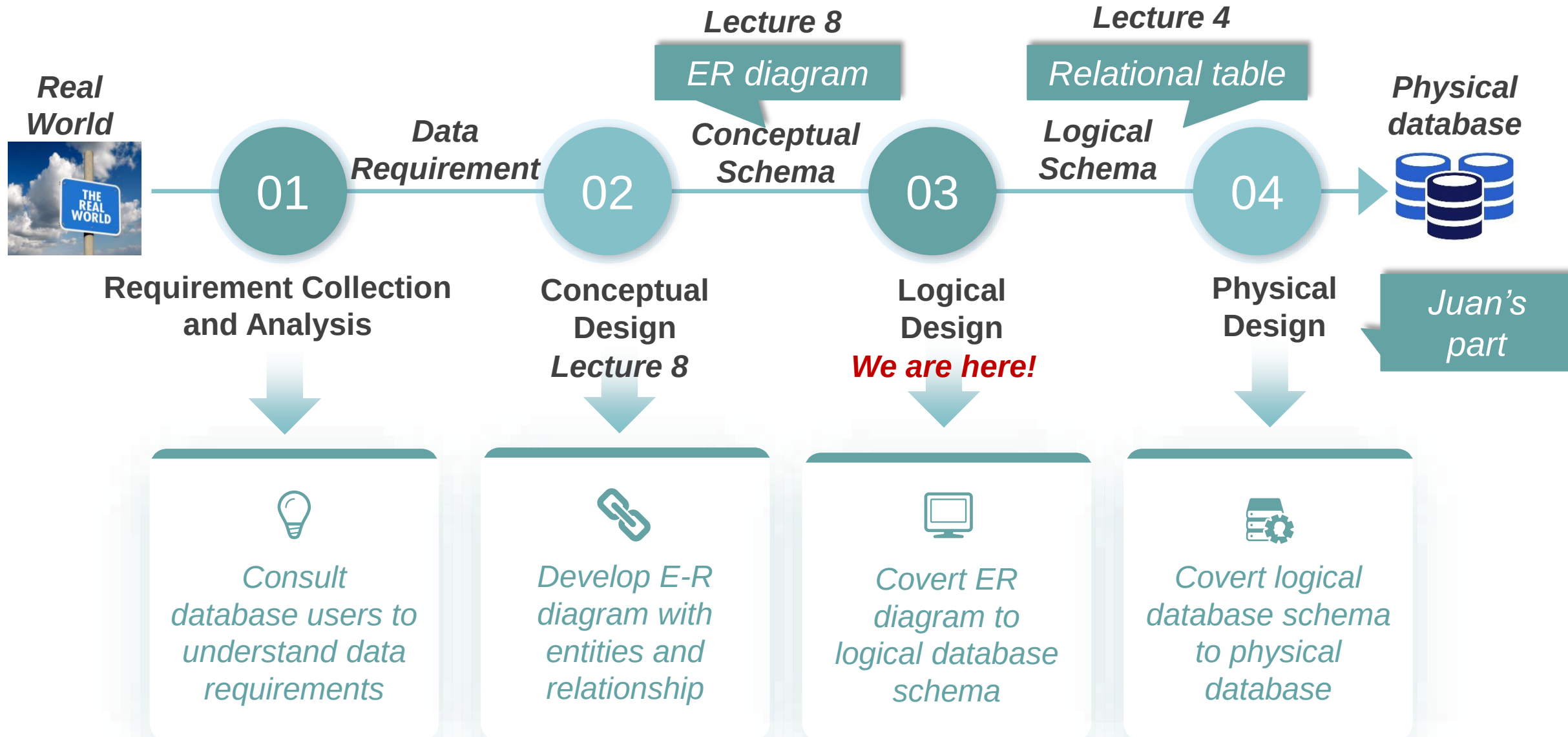
Relational Model

- Relations (tables)
- Tuples (rows)
- Attributes (columns)

✓ *Computer-understandable*

Provides a logical view of the data, focusing on implementation in DBMS.

Steps of Database Design



Agenda



- Review the ER model and relational model
- Mapping basic concepts to relations
 - Entity types
 - Relationship types
- Map complex cases to relations
 - Weak entity types
 - Recursive relationship types
 - N-ary relationship types
 - Special attributes
- Relational modelling of specialization

Review: Relation Schema



- Relation schema: the logical structure of the relation.
 - Given attributes $A_1, A_2, \dots, A_n$, $R = (A_1, A_2, \dots, A_n)$ is a relation schema.
 - Example:
 - ◆ instructor = (ID, name, dept_name, salary)
- Relation instance: a snapshot of the data in the relation at a given instant in time
 - A relation instance r defined over R is denoted by $r(R)$.
 - An element t of relation r is called a tuple and is represented by a row in a table.

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
983456	Kim	Elec. Eng.	80000



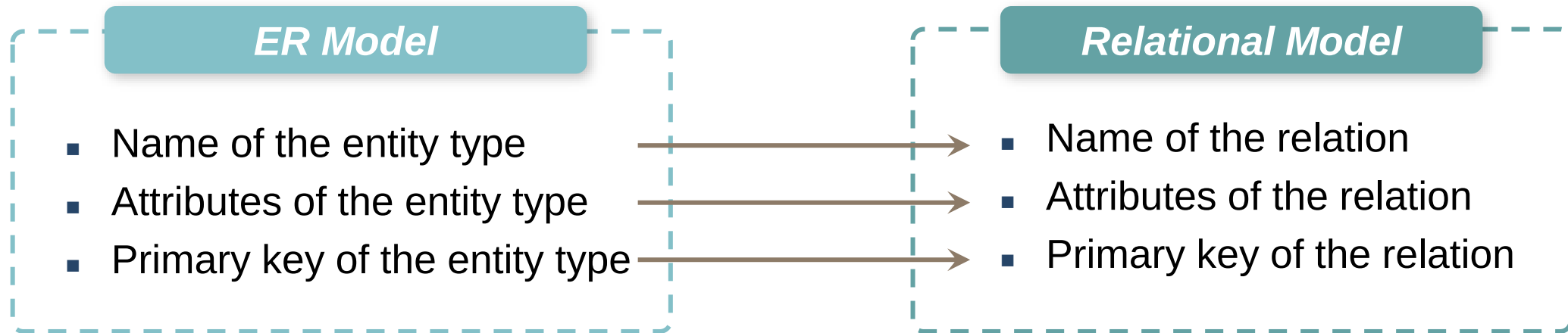
Relation instance

instructor

Mapping Entity Types to Relations



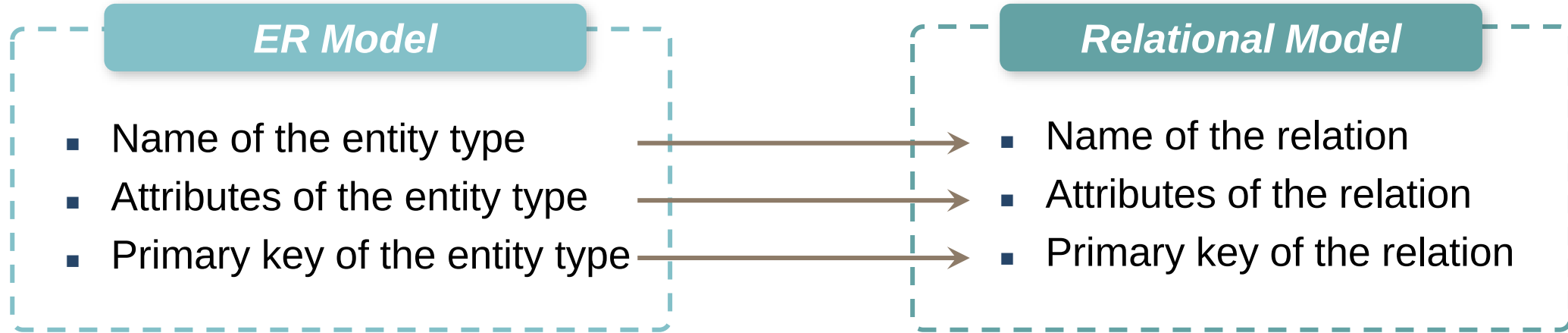
- **Create** a relation for each regular entity type.



Mapping Entity Types to Relations

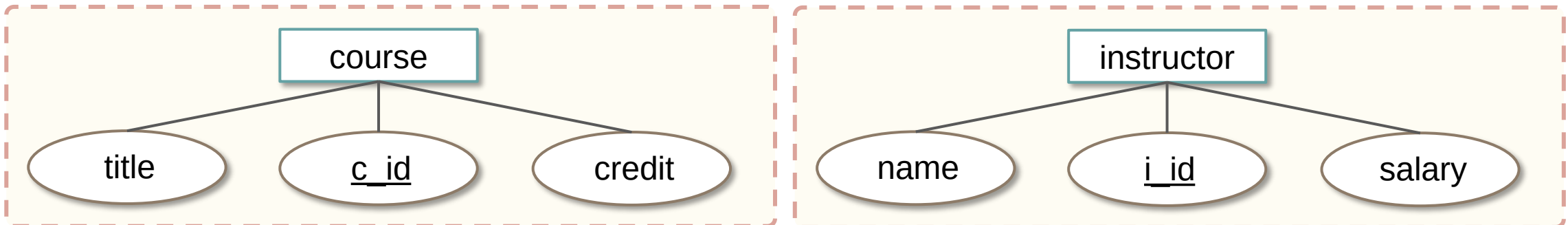


- **Create** a relation for each regular entity type.



- **Example**

- # *We do not care about the order of attributes in this context.*
- # *And for the moment we also do not care about attribute domains.*

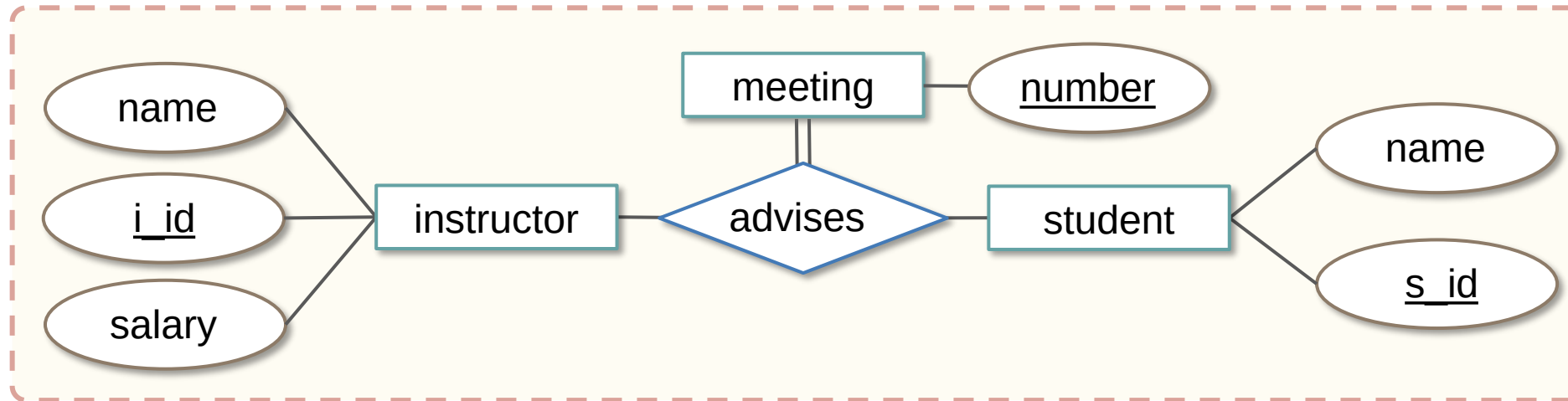


- Relation schema: *course(c_id, title, credit)*
- Relation schema: *instructor(i_id, name, salary)*

Mapping Entity Types to Relations



- Special case
 - If there are no attributes other than the primary key and the entity participates totally in a relationship, then the relation can be eliminated.
 - Example
 - ◆ In this case, meeting relation can be eliminated.
 - ◆ We can maintain the number in *advisor* relation.



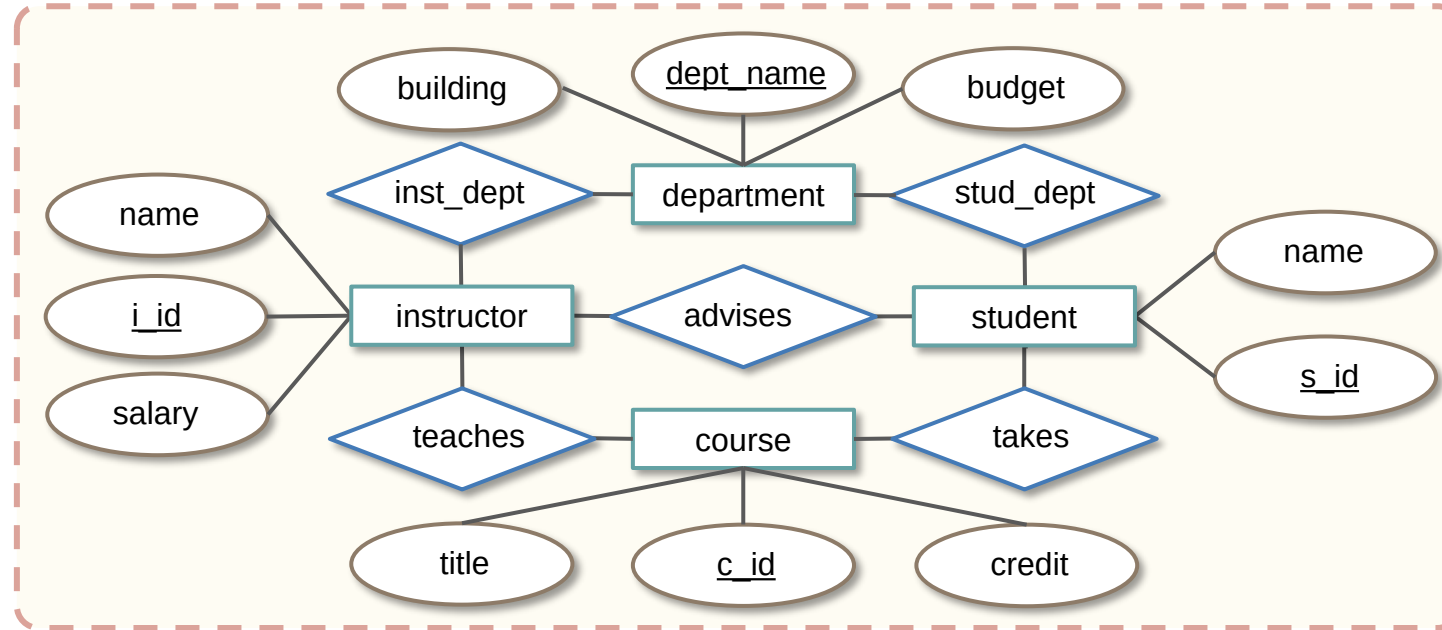
- *student*(s_id, title, credit)
- *instructor*(i_id, name, salary)
- ~~*meeting*(number)~~

advises(i_id → *instructor*, s_id → *student*, *meeting_number*)

Exercise 1



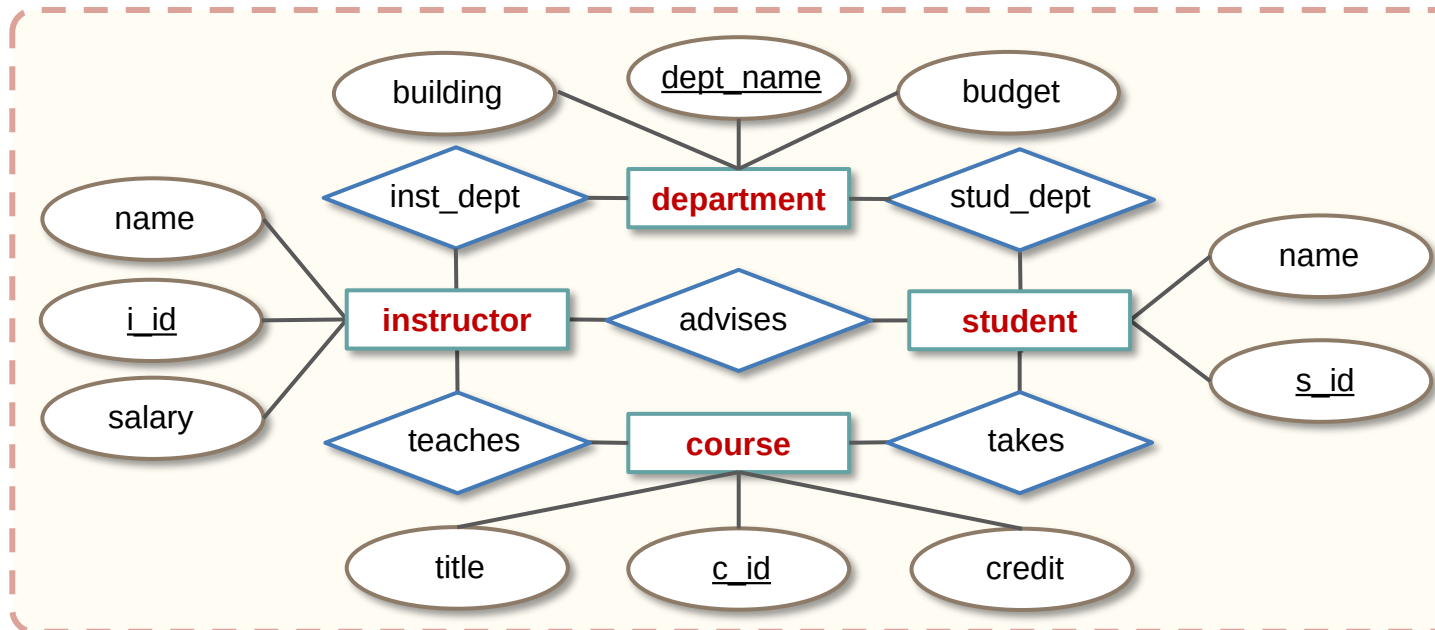
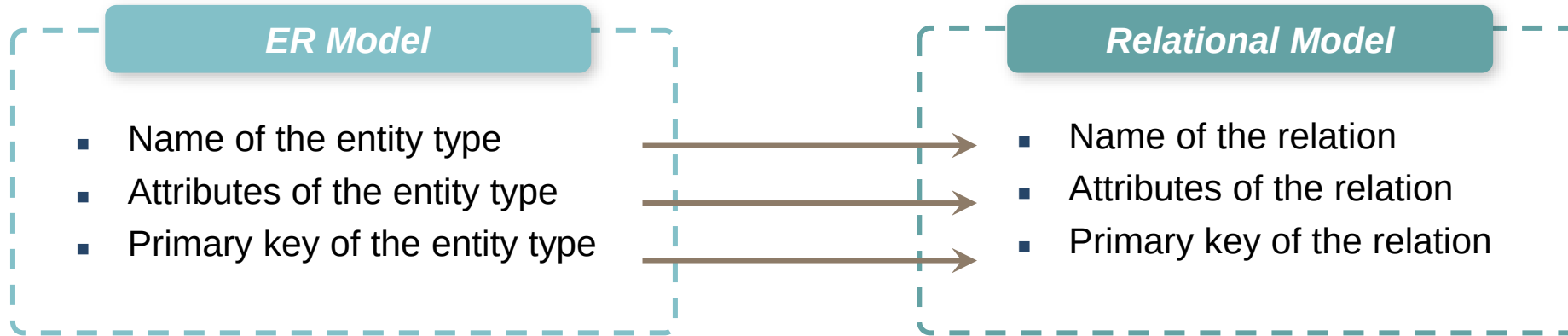
- *Create relations for entity types.*



Exercise 1 -- Solution



- *Create relations for entity types.*



- *instructor(i_id, name, salary)*
- *student(s_id, name)*
- *course(c_id, title, credit)*
- *department(dept_name, building, budget)*

Agenda

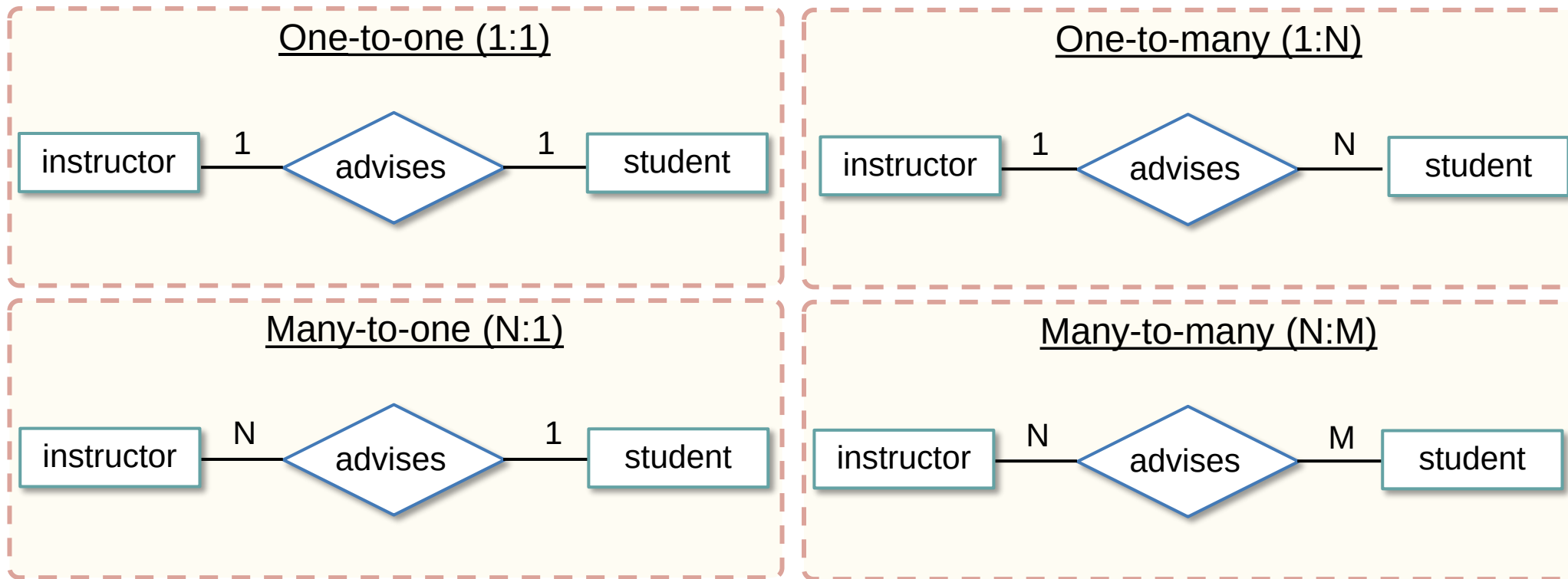


- Review the ER model and relational model
- Mapping basic concepts to relations
 - Entity types
 - Relationship types
- Map complex cases to relations
 - Weak entity types
 - Recursive relationship types
 - N-ary relationship types
 - Special attributes
- Relational modelling of specialization

Review: Cardinality Ratio

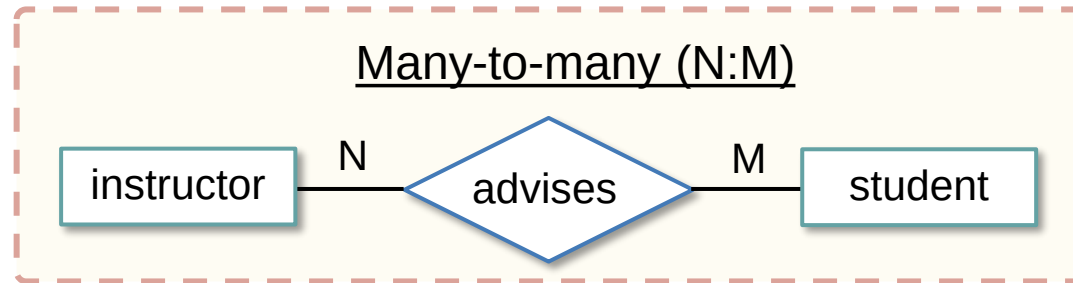


- Number of entities involved in a relationship instance
- For a binary relationship set R between entity sets A and B , the mapping cardinalities must be one of the following:



Mapping of Many-to-Many (N:M) Relationship Types

- **Create** a relation for each many-to-many (N:M) relationship type.



ER Model

- Name of the relationship type
- Attributes of the relationship type
- Primary keys of **the participating entity types**
- The **union of primary keys** of the participating entity types

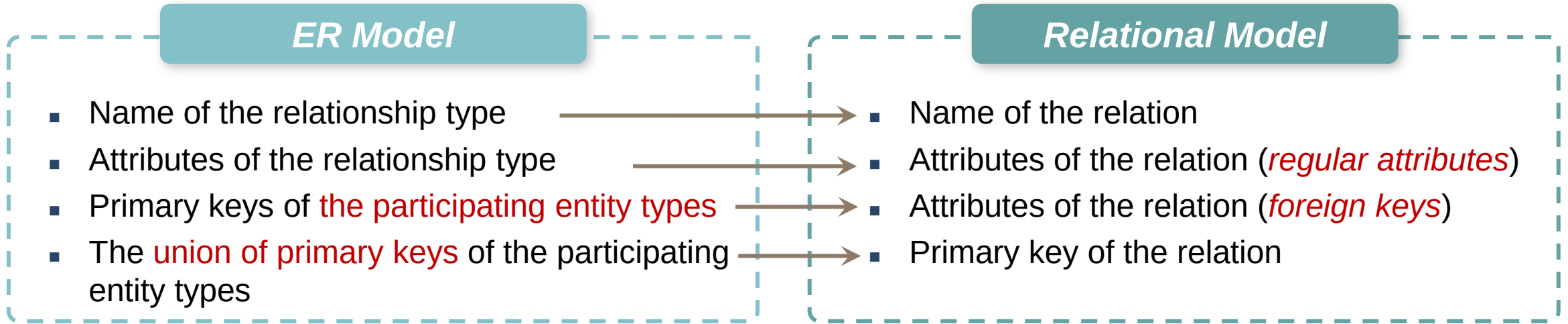
Relational Model

- Name of the relation
- Attributes of the relation (**regular attributes**)
- Attributes of the relation (**foreign keys**)
- Primary key of the relation

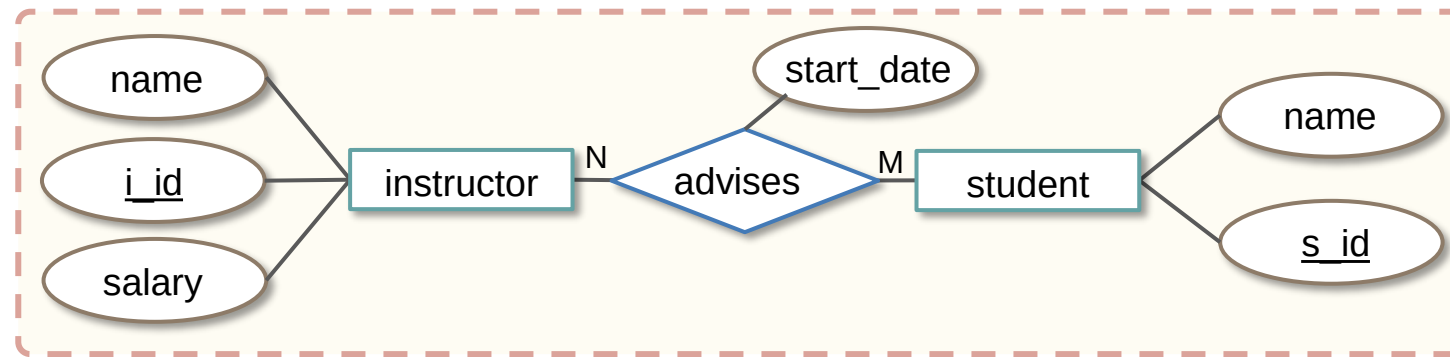
Attributes of the relation = attributes of the relationship + primary keys of the participating entities

Mapping of Many-to-Many (N:M) Relationship Types

- **Create** a relation for each many-to-many (N:M) relationship type.



- Example:

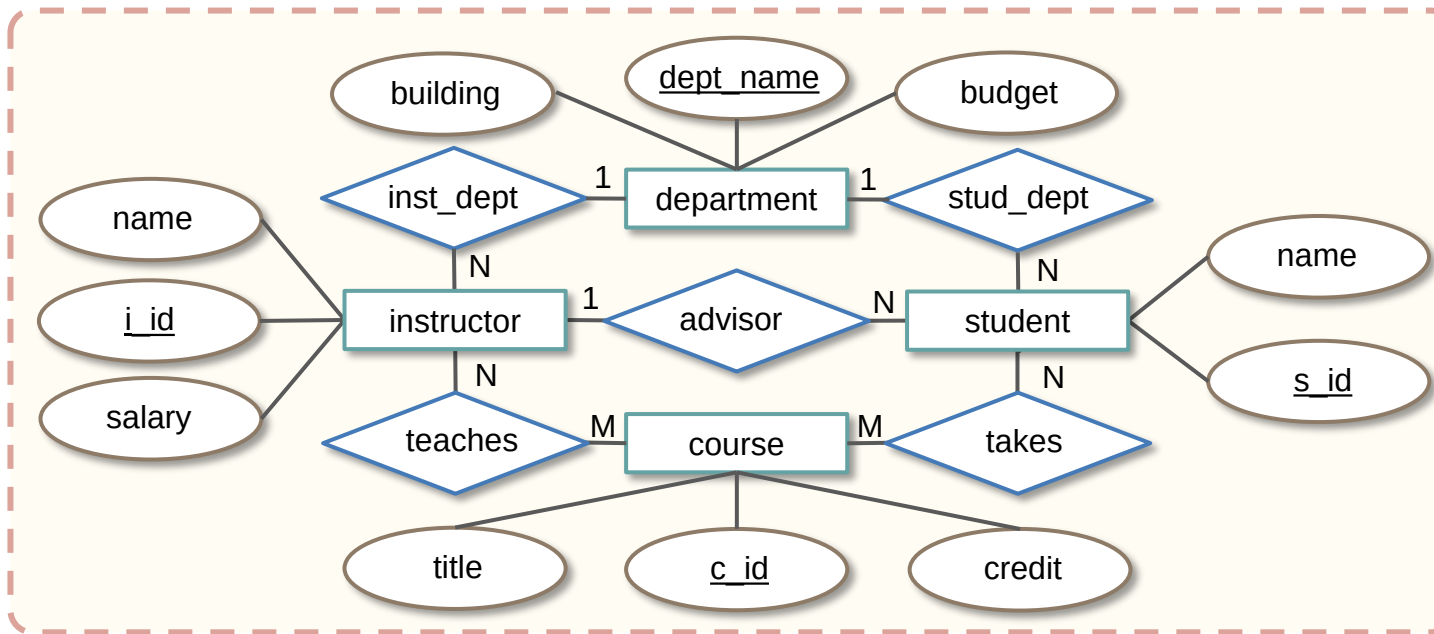


- $advises(\underline{i_id} \rightarrow instructor, \underline{s_id} \rightarrow student, start_date)$

Exercise 2



- Identify all many-to-many (N:M) relationship types in the ER diagram and create relations for them.

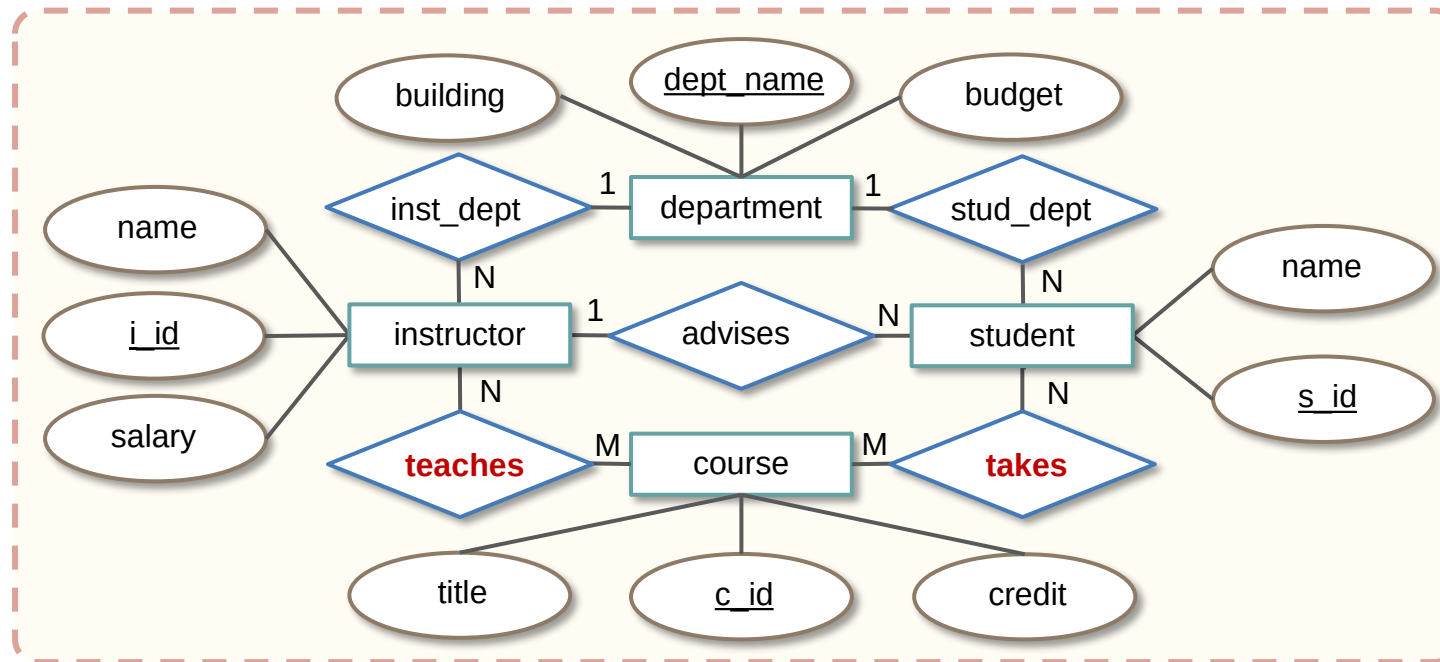


- `instructor(i_id, name, salary)`
- `student(s_id, name)`
- `course(c_id, title, credit)`
- `department(dept_name, building, budget)`

Exercise 2 -- Solution



- *Identify all many-to-many (N:M) relationship types in the ER diagram and create relations for them.*



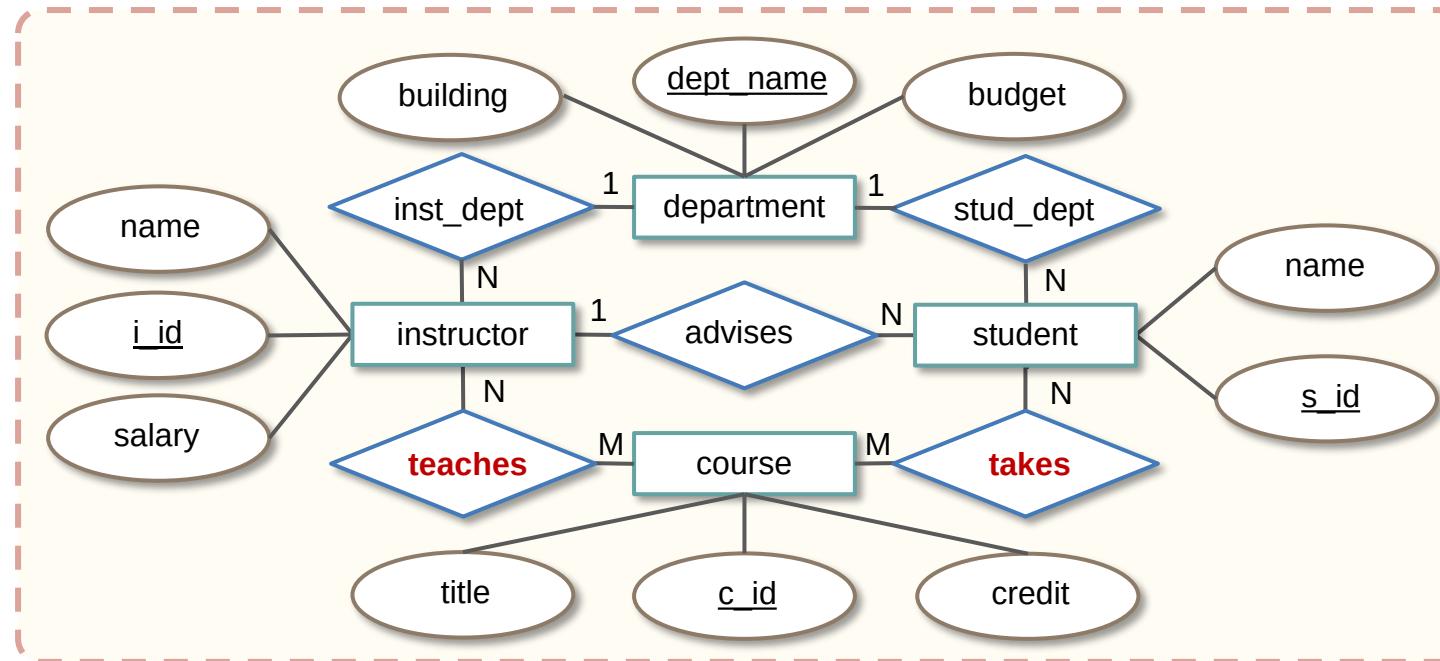
*Many-to-many (N:M)
relationship*

- *teaches*
- *takes*

Exercise 2 -- Solution



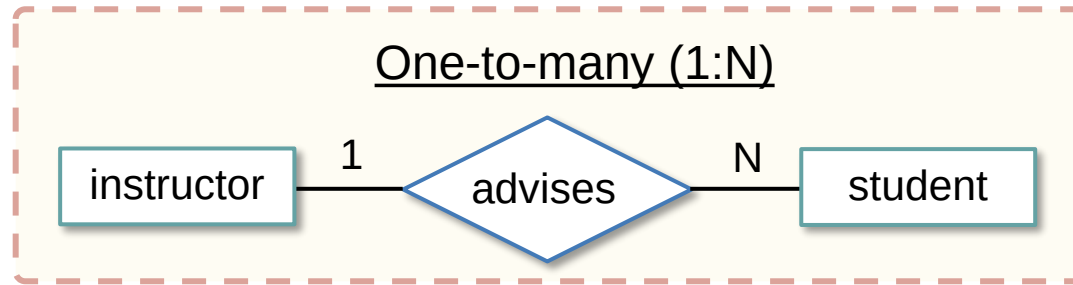
- Identify all many-to-many (N:M) relationship types in the ER diagram and create relations for them.



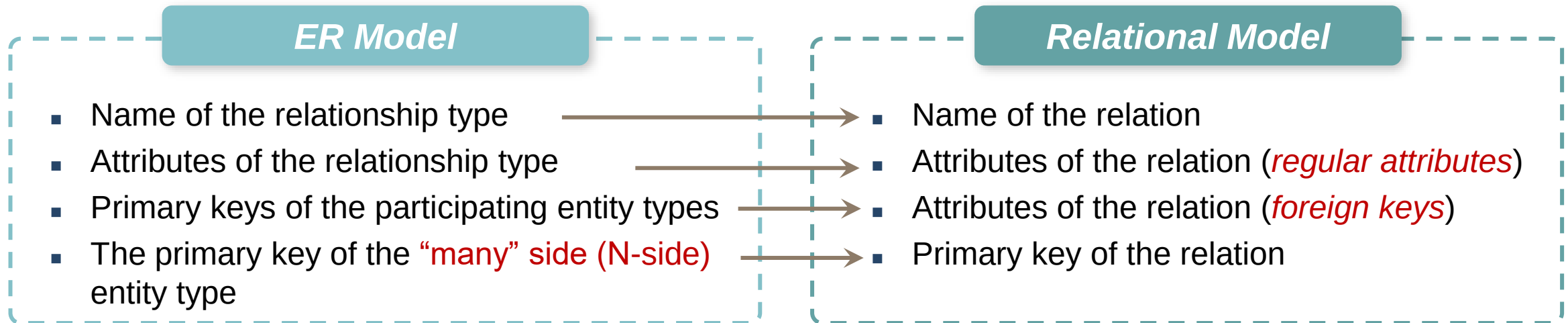
Many-to-many (N:M)

- $teaches(\underline{i_id} \rightarrow instructor, \underline{c_id} \rightarrow course)$
- $takes(\underline{s_id} \rightarrow student, \underline{c_id} \rightarrow course)$

Mapping of One-to-Many (1:N) Relationship Types



- For each one-to-many (1:N) relationship type, there are two options.
 - Option 1: **Create** a separate table for the relationship.



Mapping of One-to-Many (1:N) Relationship Types

- Option 1: **Create** a separate table for the relationship.

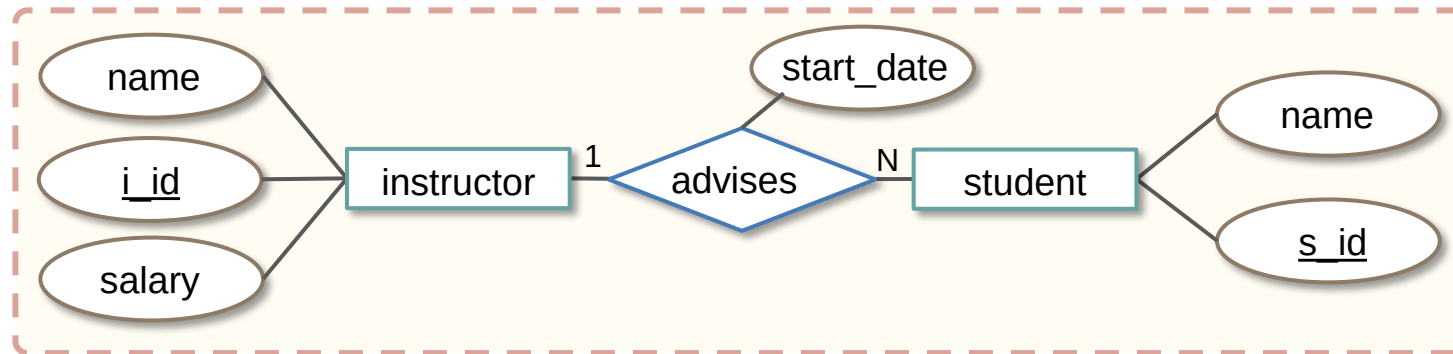
ER Model

- Name of the relationship type
- Attributes of the relationship type
- Primary keys of **the participating entity types**
- The primary key of the **“many” side (N-side)** entity type

Relational Model

- Name of the relation
- Attributes of the relation (**regular attributes**)
- Attributes of the relation (**foreign keys**)
- Primary key of the relation

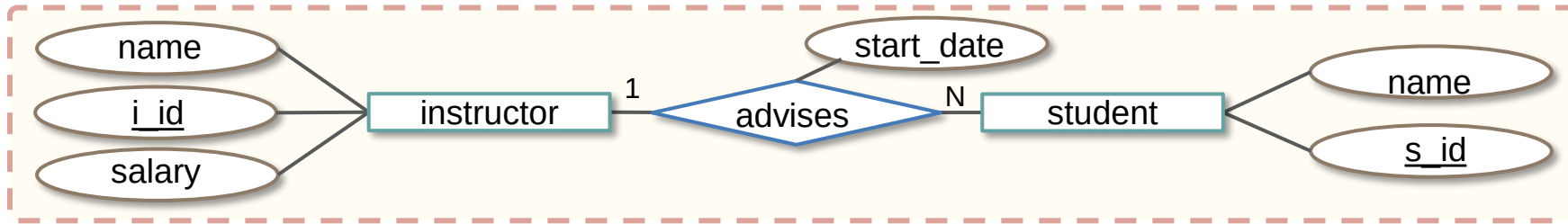
- Example:



- advises(i_id → instructor, s_id → student, start_date)***

Mapping of One-to-Many (1:N) Relationship Types

- Option 1: **Create** a separate table for the relationship.



<u>i_id</u>	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

instructor

<u>s_id</u>	name
1001	Alice
1002	Anna
1003	Bob
1004	Cathy
1005	Chris
1006	David

student

<u>s_id</u>	<u>i_id</u>	start_date
1001	12121	Mar 10
1002	33456	Sep 3
1003	22222	Sep 3
1004	22222	Sep 3
1005	76766	Sep 3
1006	45565	Mar 10

*advises(i_id → instructor,
s_id → student, start_date)*

Mapping of One-to-Many (1:N) Relationship Types

- Option 2: Improvement by merging
 - In a 1:N relationship, each tuple on the “many” side corresponds to **at most one** tuple on the “one” side.
 - The **primary key of the “many” side** is already sufficient to uniquely identify the relationship.
 - The **foreign key from the “one” side** can be added directly into the “many” side table.
 - Relationship attributes** can also be stored in the “many” side table without redundancy.

<u>i_id</u>	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

“One” side instructor

<u>s_id</u>	name
1001	Alice
1002	Anna
1003	Bob
1004	Cathy
1005	Chris
1006	David

student
“Many” side

?

+

<u>s_id</u>	<u>i_id</u>	<u>start_date</u>
1001	12121	Mar 10
1002	33456	Sep 3
1003	22222	Sep 3
1004	22222	Sep 3
1005	76766	Sep 3
1006	45565	Mar 10

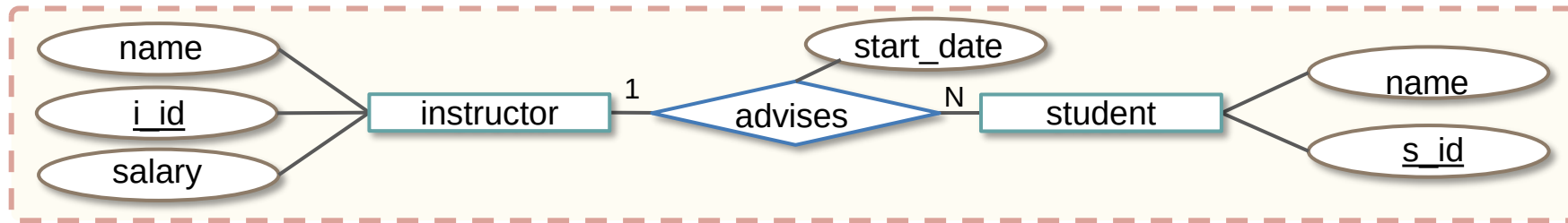
advises(*i_id* → *instructor*, *s_id*
→ *student*, *start_date*)

- Both tables share the same primary key *s_id*.
- The relationship is 1:N, so each student is linked to at most one instructor

Can we merge the “many” side (N-side) with the relationship?

Mapping of One-to-Many (1:N) Relationship Types

- Option 2: Improvement by merging



<u>i_id</u>	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

instructor
"One" side

<u>s_id</u>	name
1001	Alice
1002	Anna
1003	Bob
1004	Cathy
1005	Chris
1006	David

student "Many" side

+

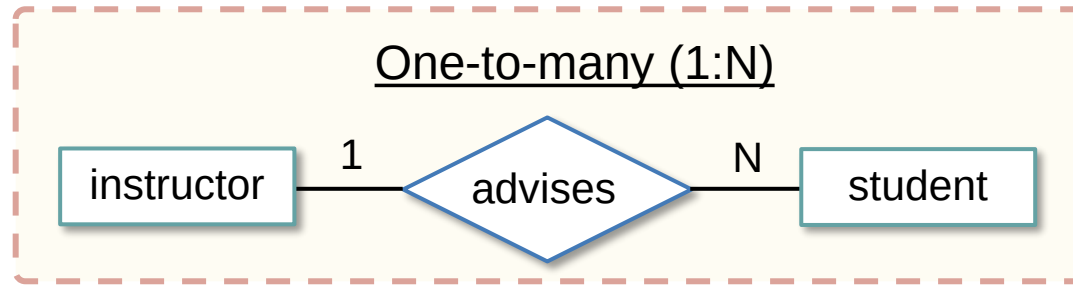
<u>s_id</u>	<u>i_id</u>	start_date
1001	12121	Mar 10
1002	33456	Sep 3
1003	22222	Sep 3
1004	22222	Sep 3
1005	76766	Sep 3
1006	45565	Mar 10

advises(*i_id* → *instructor*, *s_id* → *student*, *start_date*)

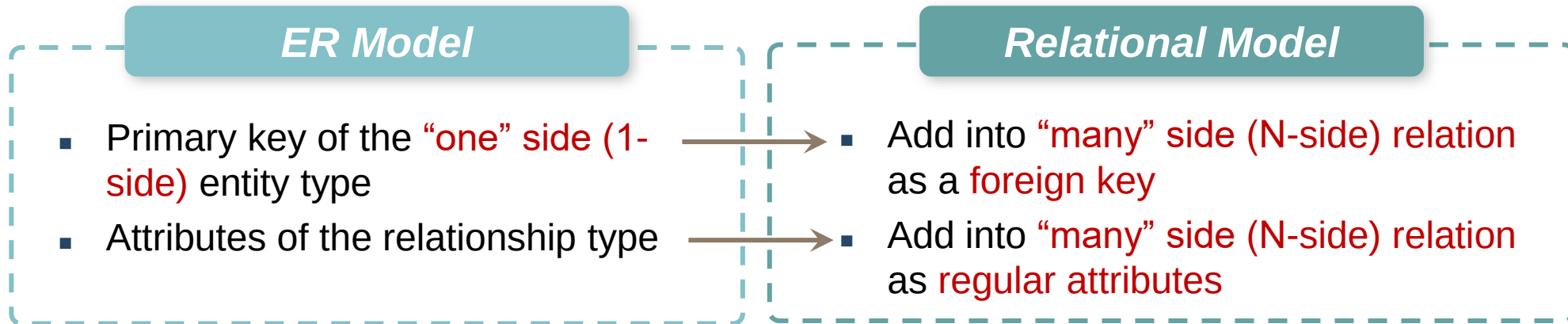
<u>s_id</u>	name	<u>i_id</u>	start_date
1001	Alice	12121	Mar 10
1002	Anna	33456	Sep 3
1003	Bob	22222	Sep 3
1004	Cathy	22222	Sep 3
1005	Chris	76766	Sep 3
1006	David	45565	Mar 10

student(*s_id*, *name*, *i_id* → *instructor*, *start_date*)

Mapping of One-to-Many (1:N) Relationship Types



- For each one-to-many (1:N) relationship type, there are two options.
 - Option 2: **Extend** the “many” side (N-side) table. (Improved)



Mapping of One-to-Many (1:N) Relationship Types

- Option 2: **Extend** the “many” side (N-side) table. (Improved)

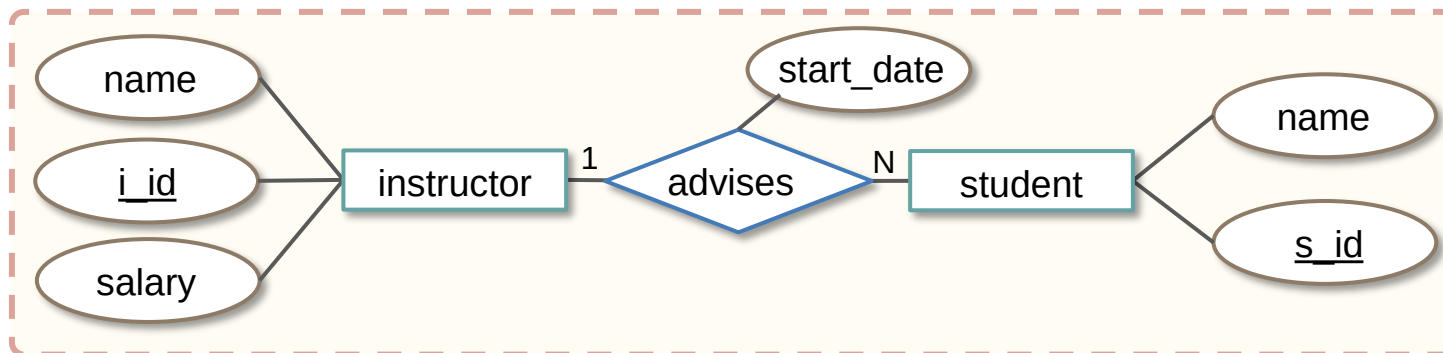
ER Model

- Primary key of the “one” side (1-side) entity type
- Attributes of the relationship type

Relational Model

- Add into “many” side (N-side) relation as a foreign key
- Add into “many” side (N-side) relation as regular attributes

Example:



Rename attributes if needed:

- student(s_id, name, advisedBy_id → instructor, advisor_start_date)*

“One” side ■ *instructor(i_id, name, salary)*

“Many” side ■ *student(s_id, name)* → ■ *student(s_id, name, i_id → instructor, start_date)*

Mapping of One-to-Many (1:N) Relationship Types

- Option 2: **Extend** the “many” side (N-side) table.
 - If the participation is not total, merging requires null values for the foreign key. In such cases, it might be preferable for some applications to have a separate relation.
 - Example:

<u>i_id</u>	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

instructor
“One” side

<u>s_id</u>	name
1001	Alice
1002	Anna
1003	Bob
1004	Cathy
1005	Chris
1006	David

student “Many” side

+

<u>s_id</u>	<u>i_id</u>	start_date
1001	12121	Mar 10
1002	33456	Sep 3
1003	22222	Sep 3
1004	22222	Sep 3
1005	76766	Sep 3
1006	45565	Mar 10

advises(i_id → *instructor*, s_id → *student*, *start_date*)

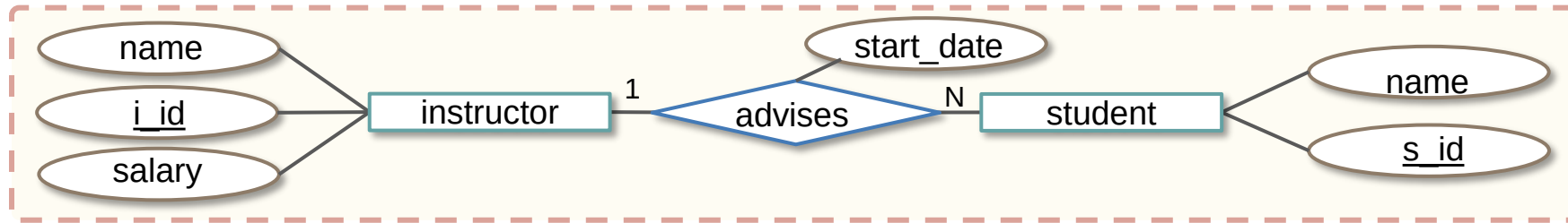
<u>s_id</u>	name	<u>i_id</u>	start_date
1001	Alice	12121	Mar 10
1002	Anna	33456	Sep 3
1003	Bob	22222	Sep 3
1004	Cathy	22222	Sep 3
1005	Chris	76766	Sep 3
1006	David	Null	Null

student(s_id, *name*, i_id → *instructor*, *start_date*)

Mapping of One-to-Many (1:N) Relationship Types

- Option 2: **Extend** the “many” side (N-side) table.

Why not “one” side (1-side)?



<u>i_id</u>	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

instructor “One” side

?

+

<u>s_id</u>	i_id	start_date
1001	12121	Mar 10
1002	33456	Sep 3
1003	22222	Sep 3
1004	22222	Sep 3
1005	76766	Sep 3
1006	45565	Mar 10

advises(i_id → instructor, s_id
→ student, start_date)

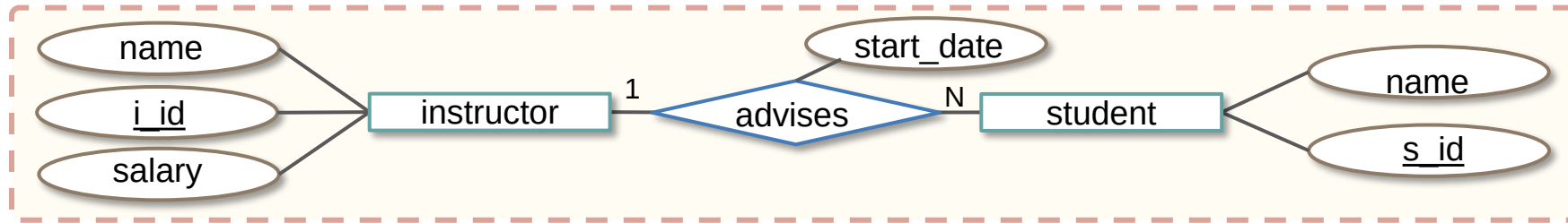
<u>s_id</u>	name
1001	Alice
1002	Anna
1003	Bob
1004	Cathy
1005	Chris
1006	David

student
“Many” side

Mapping of One-to-Many (1:N) Relationship Types

- Option 2: **Extend** the “many” side (N-side) table.

Why not “one” side (1-side)?

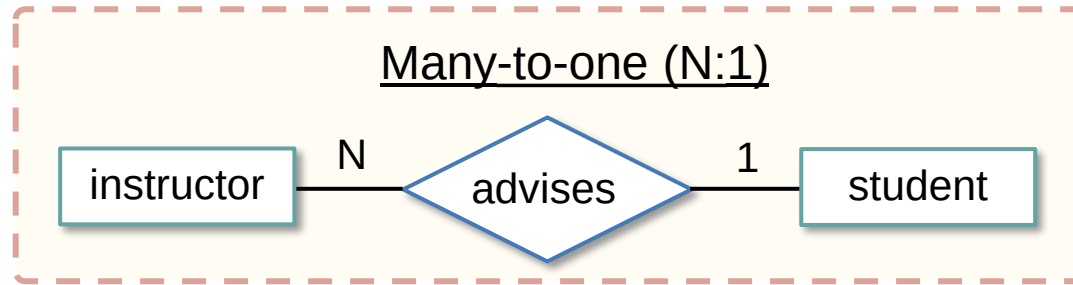


- Update anomaly:
 - What happens when Einstein's salary updated?
- Deletion anomaly
 - What happens if Einstein no longer taught?
- Insert anomaly:
 - Curie is new and does not yet advise any students.

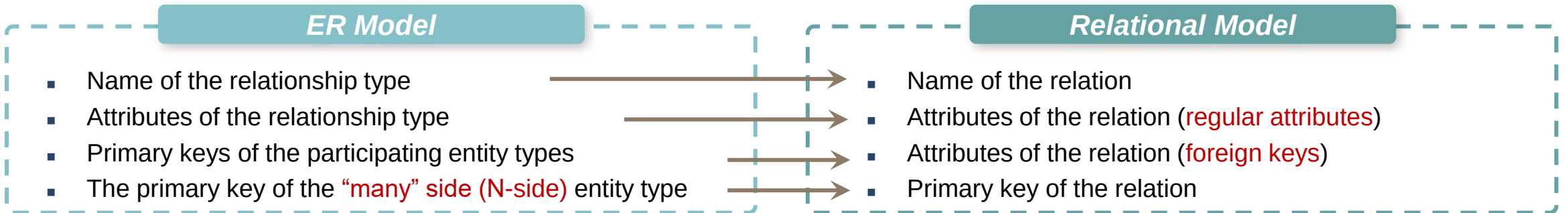
<u>i_id</u>	name	salary	<u>s_id</u>	start_date
10101	Srinivasan	65000	NULL	NULL
12121	Wu	90000	1001	Mar 10
15151	Mozart	40000	NULL	NULL
22222	Einstein	95000	1003	Sep 3
22222	Einstein	95000	1004	Sep 3
32343	El Said	60000	NULL	NULL
33456	Gold	87000	1002	Sep 3
45565	Katz	75000	1006	Mar 10
58583	Califieri	62000	NULL	NULL
76543	Singh	80000	NULL	NULL
76766	Crick	72000	1005	Sep 3
83821	Brandt	92000	NULL	NULL
98345	Kim	80000	NULL	NULL

instructor

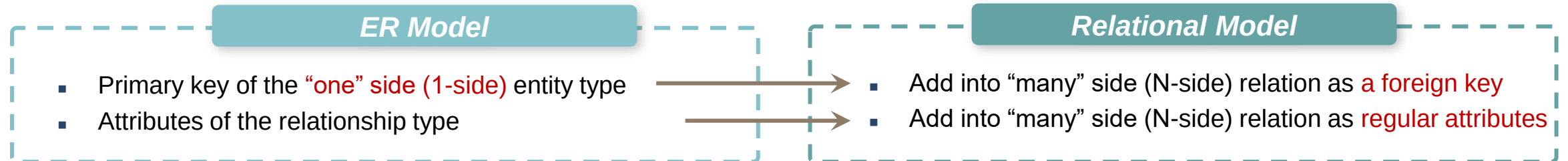
Mapping of Many-to-One (N:1) Relationship Types



- Similar to the mapping of a one-to-many (1:N) relationship
 - Option 1: **Create** a separate table for the relationship.



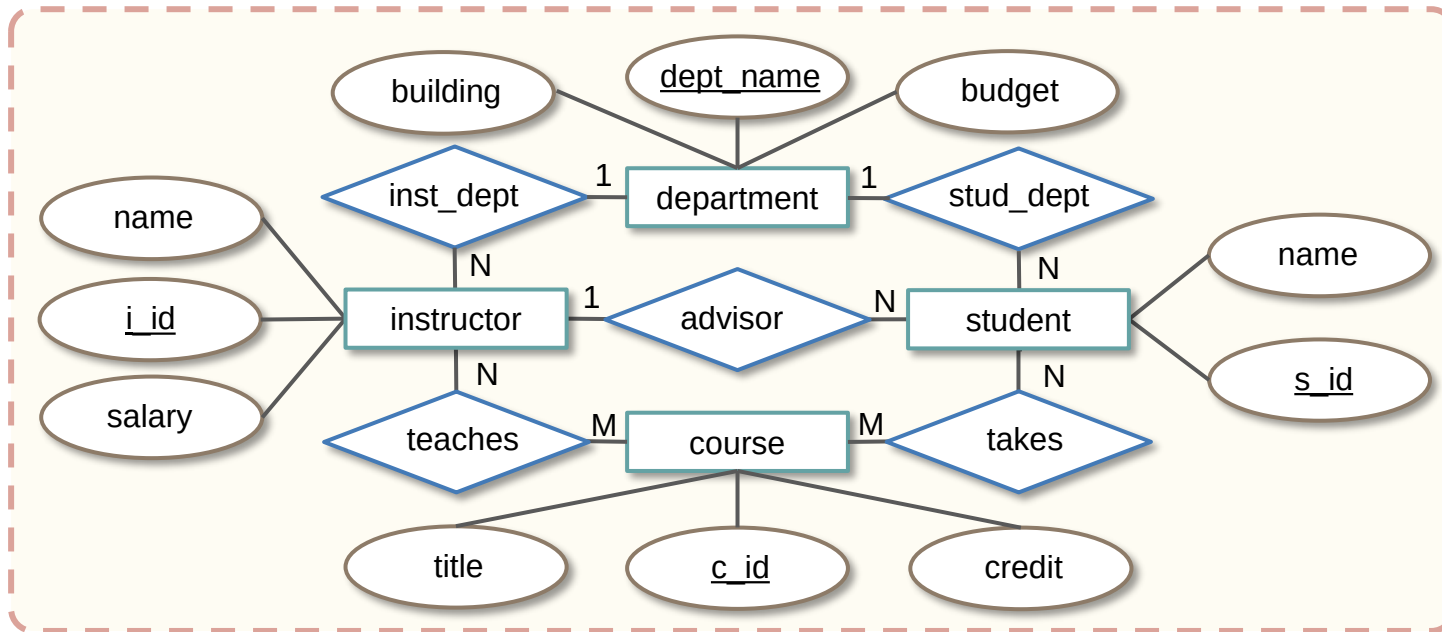
- Option 2: **Extend** the “many” side (N-side) table.



Exercise 3



- *Identify all one-to-many (1:N) relationship types in the diagram*
- *Create these relations for relationship types or revise the existing relations for entity types.*

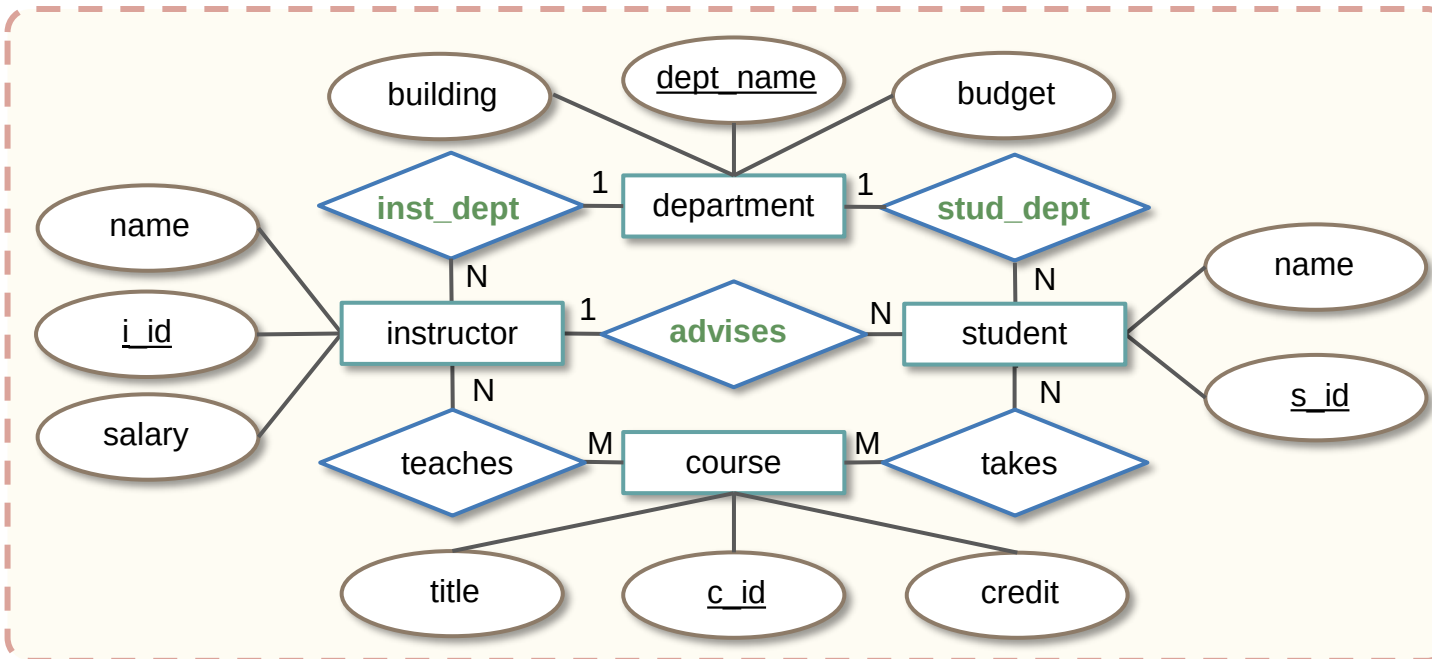


- `instructor(i_id, name, salary)`
- `student(s_id, name)`
- `course(c_id, title, credit)`
- `department(dept_name, building, budget)`

Exercise 3 -- Solution



- *Identify all one-to-many (1:N) relationship types in the diagram*
- *Create these relations for relationship types or revise the existing relations for entity types.*



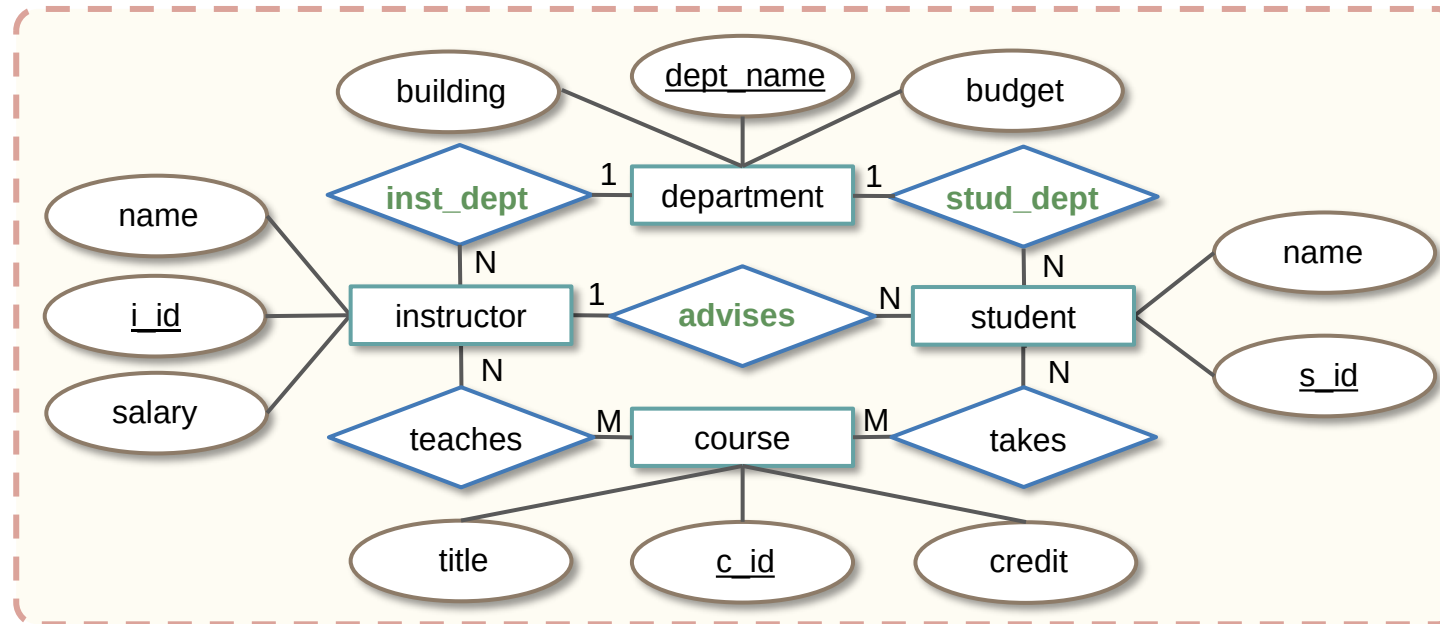
*One-to-many (1:N)
relationship*

- *inst_dept*
- *stud_dept*
- *advises*

Exercise 3 -- Solution



- Create these relations for relationship types or revise the existing relations for entity types.



One-to-many (1:N)

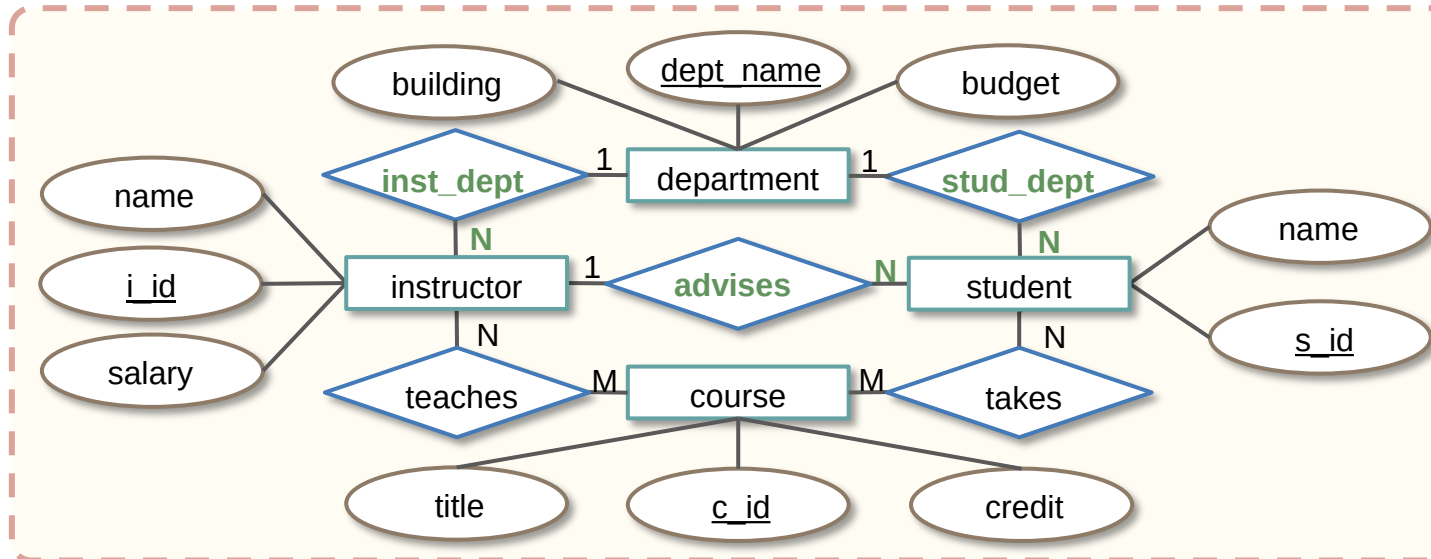
Option 1:

- `inst_dept(i_id → instructor, dept_name → department)`
- `stud_dept(s_id → student, dept_name → department)`
- `advises (s_id → student, i_id → instructor )`

Exercise 3 -- Solution



- Create these relations for relationship types or revise the existing relations for entity types.



- ***instructor***(*i_id*, *name*, *salary*)
- ***student***(*s_id*, *name*)
- *course*(*c_id*, *title*, *credit*)
- *department*(*dept_name*, *building*, *budget*)

One-to-many (1:N)

Option 2:

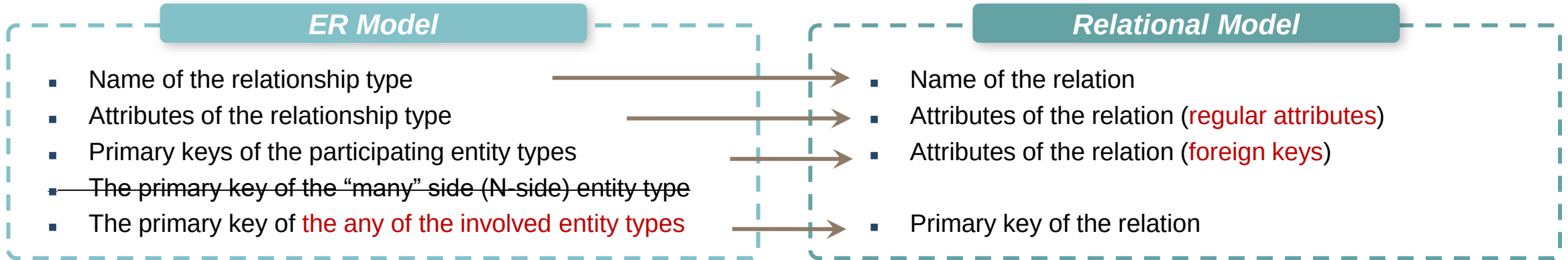
- *instructor*(*i_id*, *name*, *salary*, *dept_name* → *department*)
- *student*(*s_id*, *name*, *dept_name* → *department*)
- *student*(*s_id*, *name*, *dept_name* → *department*, *i_id* → *instructor*)



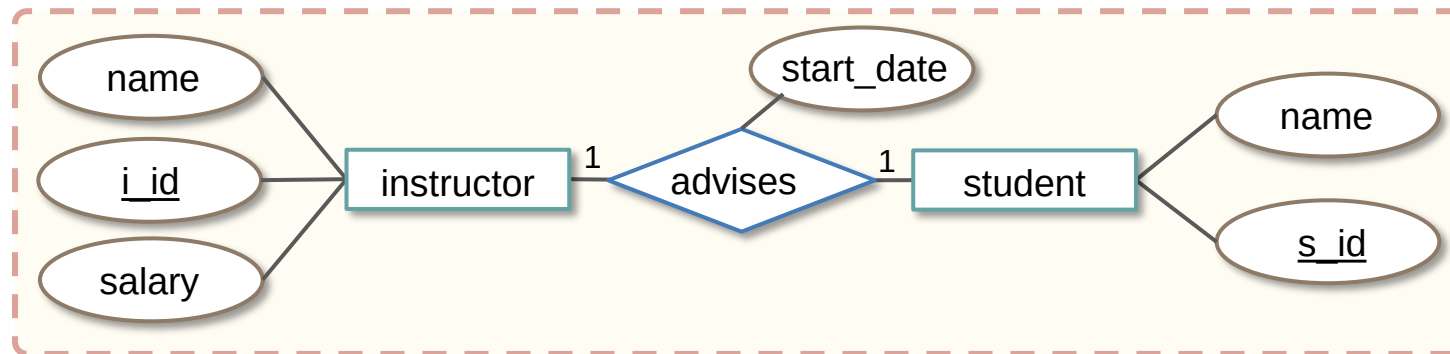
Mapping of One-to-One (1:1) Relationship Types



- Is this different from 1:N relationship types?
 - Option 1: **Create** a separate table for the relationship.



- Example



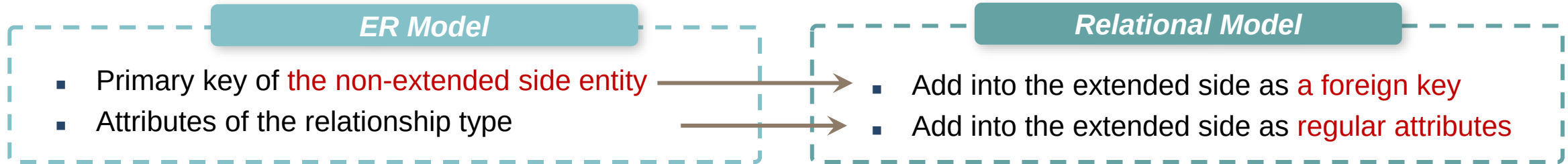
- $advises(\underline{i_id} \rightarrow instructor, \underline{s_id} \rightarrow student, start_date)$
or
- $advises(i_id \rightarrow instructor, \underline{s_id} \rightarrow student, start_date)$

Mapping of One-to-One (1:1) Relationship Types

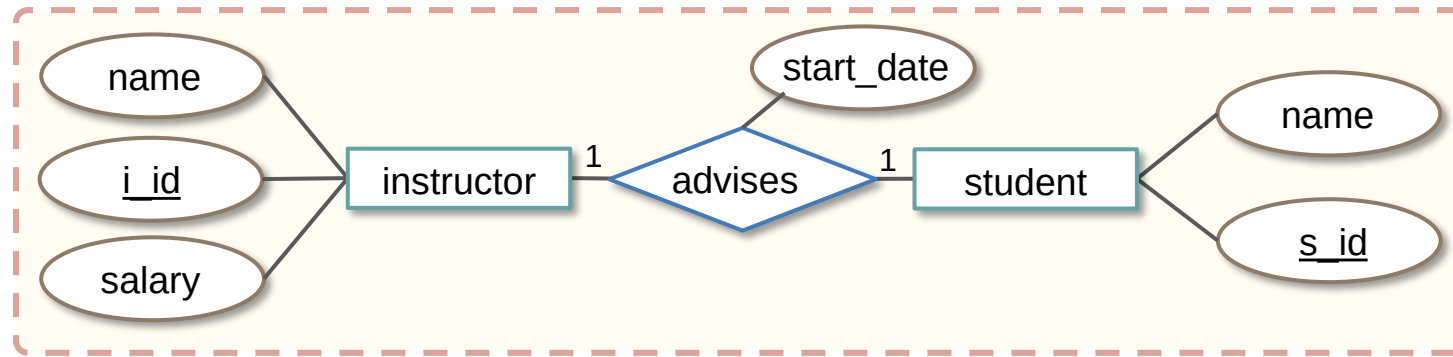


- Is this different from one-to-many (1:N) relationship types?

- ~~Option 2 (one-to-many): Extend the “many” side (N-side) table.~~
- Option 2 (one-to-one): Extend **either side** table



- Example



Extend

Extend

- `instructor(i_id, name, salary)`
- `student(s_id, name, i_id → instructor, start_date)`

or

- `instructor(i_id, name, salary, s_id → student, start_date)`
- `student(s_id, name)`

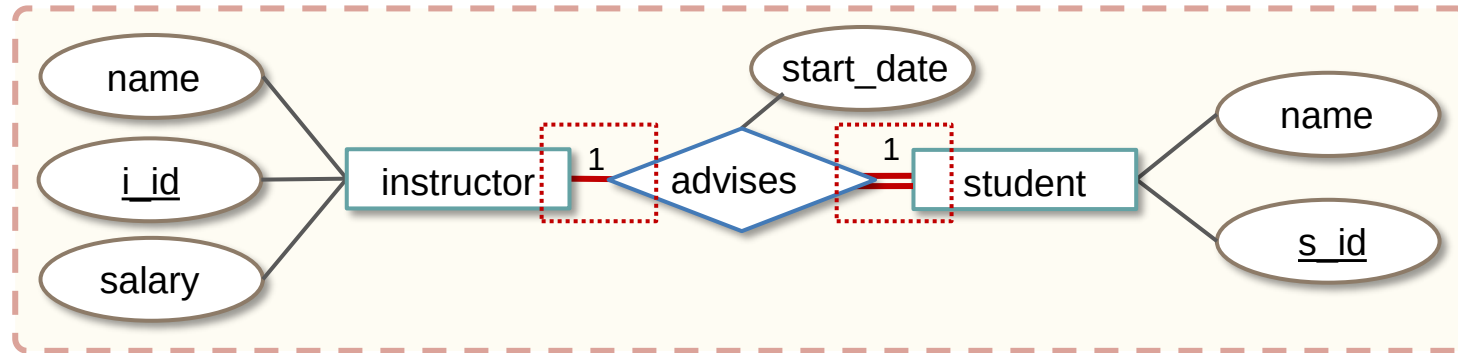
Mapping of One-to-One (1:1) Relationship Types



- Option 2: Extend **either side** table

- It is best to extend the relation of the entity type with **total participation**.

- Example



- ♦ *instructor*: partial participation
- ♦ *student*: total participation

Extend

- instructor*(*i_id*, *name*, *salary*)
- student*(*s_id*, *name*)

Extend

- instructor*(*i_id*, *name*, *salary*)
- student*(*s_id*, *name*, *i_id* → *instructor*, *start_date*)

- student* has total participation: every student must have an instructor
- No NULL values: each student row can safely store *i_id* and *start_date*

Clean and efficient schema 😊

- instructor*(*i_id*, *name*, *salary*, *s_id* → *student*, *start_date*)
- student*(*s_id*, *name*)

- instructor* has partial participation: some instructors may not advise any student
- This causes NULL values in the schema (no student for that instructor)

Less clean design ☹️

Mapping of One-to-One (1:1) Relationship Types



- Option 2: Extend **either side** table
 - It is best to extend the relation of the entity type with **total participation**.
 - Example (*extend the side with total participation*)

<u>i_id</u>	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

instructor

<u>s_id</u>	name
1001	Alice
1002	Anna
1003	Bob
1004	Cathy
1005	Chris
1006	David

student



<u>s_id</u>	<u>i_id</u>	start_date
1001	12121	Mar 10
1002	33456	Sep 3
1003	22222	Sep 3
1004	32343	Sep 3
1005	76766	Sep 3
1006	45565	Mar 10

advises

<u>s_id</u>	name	<u>i_id</u>	start_date
1001	Alice	12121	Mar 10
1002	Anna	33456	Sep 3
1003	Bob	22222	Sep 3
1004	Cathy	32343	Sep 3
1005	Chris	76766	Sep 3
1006	David	45565	Mar 10

student(s_id, name, i_id → instructor, start_date)

No NULL values

Mapping of One-to-One (1:1) Relationship Types



- Option 2: Extend **either side** table
 - It is best to extend the relation of the entity type with **total participation**.
 - Example (*extend the side with partial participation*)

Leads to NULL values

<u>i_id</u>	name	salary	s_id	start_date
10101	Srinivasan	65000	NULL	NULL
12121	Wu	90000	1001	Mar 10
15151	Mozart	40000	NULL	NULL
22222	Einstein	95000	1003	Sep 3
32343	El Said	60000	1004	Mar 10
33456	Gold	87000	1002	Sep 3
45565	Katz	75000	1006	Mar 10
58583	Califieri	62000	NULL	NULL
76543	Singh	80000	NULL	NULL
76766	Crick	72000	1005	Sep 3
83821	Brandt	92000	NULL	NULL
98345	Kim	80000	NULL	NULL

instructor(*i_id*, *name*, *salary*, *s_id* → *student*, *start_date*)



<u>i_id</u>	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45565	Katz	75000
58583	Califieri	62000
76543	Singh	80000
76766	Crick	72000
83821	Brandt	92000
98345	Kim	80000

instructor

<u>s_id</u>	<u>i_id</u>	start_date
1001	12121	Mar 10
1002	33456	Sep 3
1003	22222	Sep 3
1004	32343	Sep 3
1005	76766	Sep 3
1006	45565	Mar 10

advises

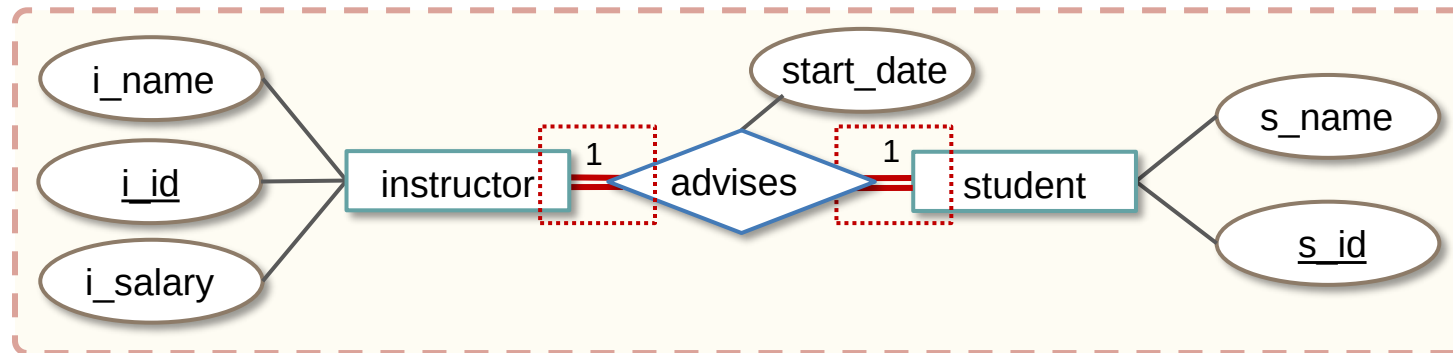
<u>s_id</u>	name
1001	Alice
1002	Anna
1003	Bob
1004	Cathy
1005	Chris
1006	David

student

Mapping of One-to-One (1:1) Relationship Types



- Option 2: Extend **either side** table
 - If **both sides have total participation**, they can be merged into **a single relation** that includes:
 - ◆ All attributes from both entities and the relationship
 - ◆ The primary key of either entity
 - Example



- ◆ *instructor*: total participation
- ◆ *student*: total participation

- *instructor*(*i_id*, *i_name*, *i_salary*)
- *student*(*s_id*, *s_name*)



■ *inst_stud*(*s_id*, *s_name*, *i_id*, *i_name*, *i_salary*, *start_date*)

or

■ *inst_stud*(*s_id*, *s_name*, *i_id*, *i_name*, *i_salary*, *start_date*)

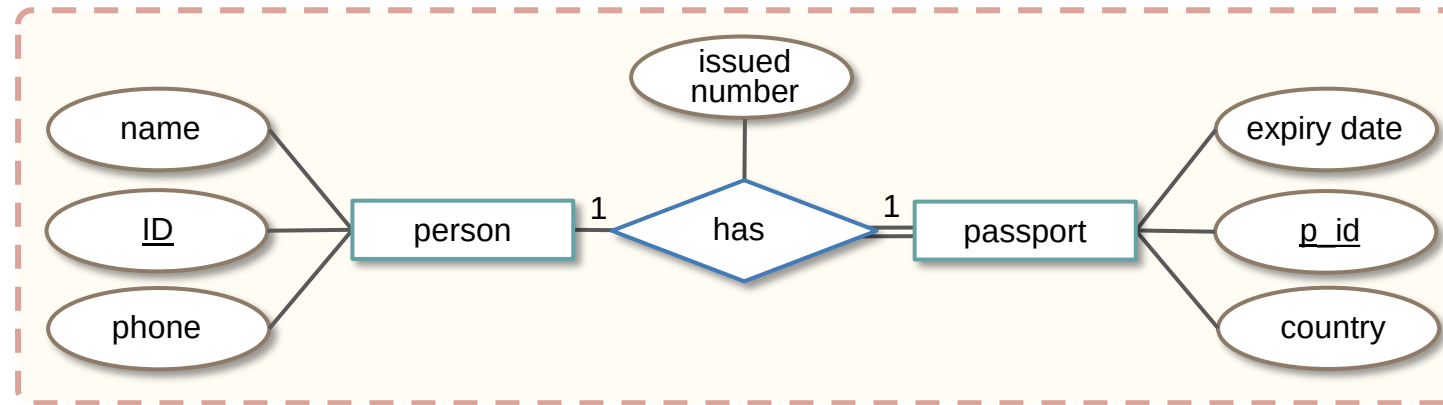
- This guarantees no NULL values when combining the entities and relationship into a single relation.

Exercise 4

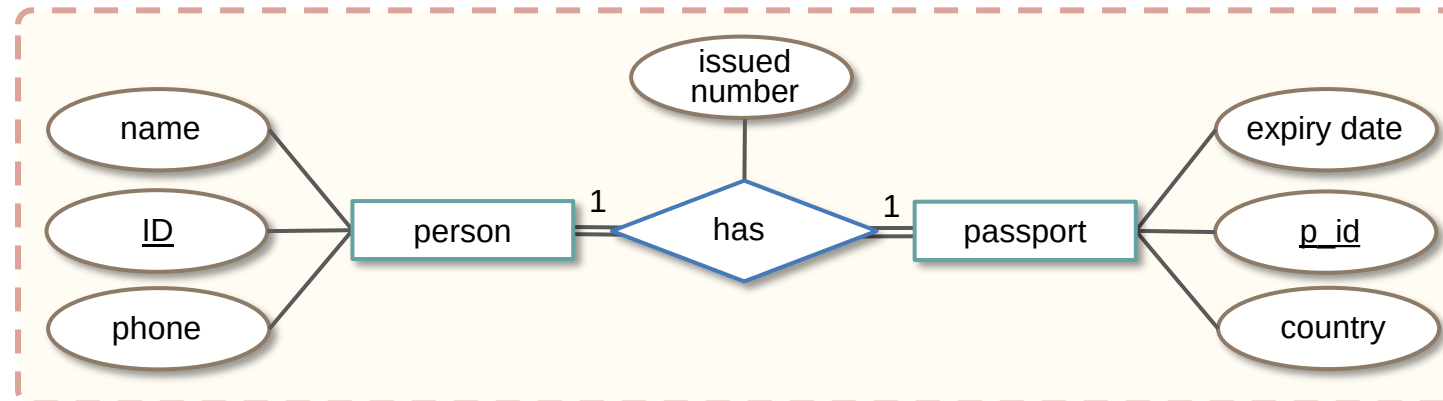


- Create relations for the ER diagram, covering both entity types and relationship types.
 - For one-to-one (1:1) relationships, apply Option 2: Extend either side table.
 - If both sides have total participation, Option 2 also allows merging them into a single relation.

D1.



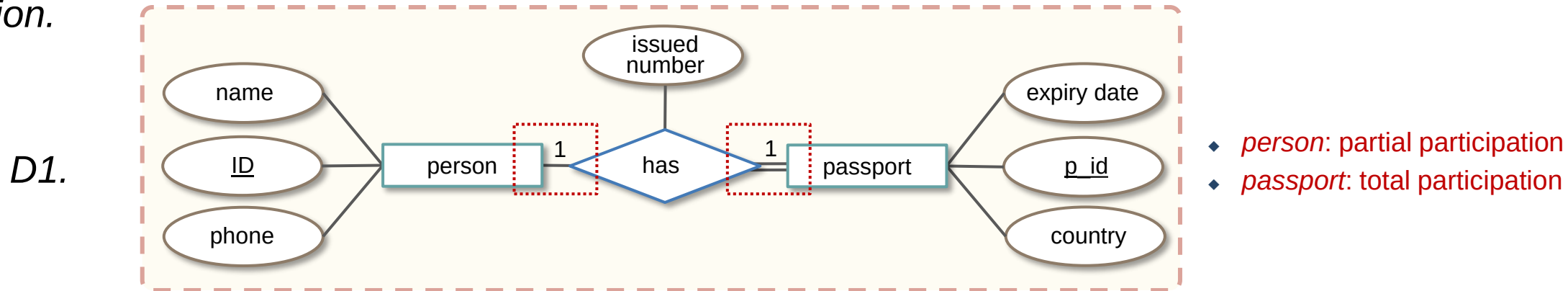
D2.



Exercise 4 -- Solution



- Create relations for the ER diagram, covering both entity types and relationship types.
 - For one-to-one (1:1) relationships, apply Option 2: Extend either side table.
 - If both sides have total participation, Option 2 also allows merging them into a single relation.



- *person*(ID, name, phone)
- *passport*(p_id, expiry date, country)

Extend the type with
total participation

Schema (D1):

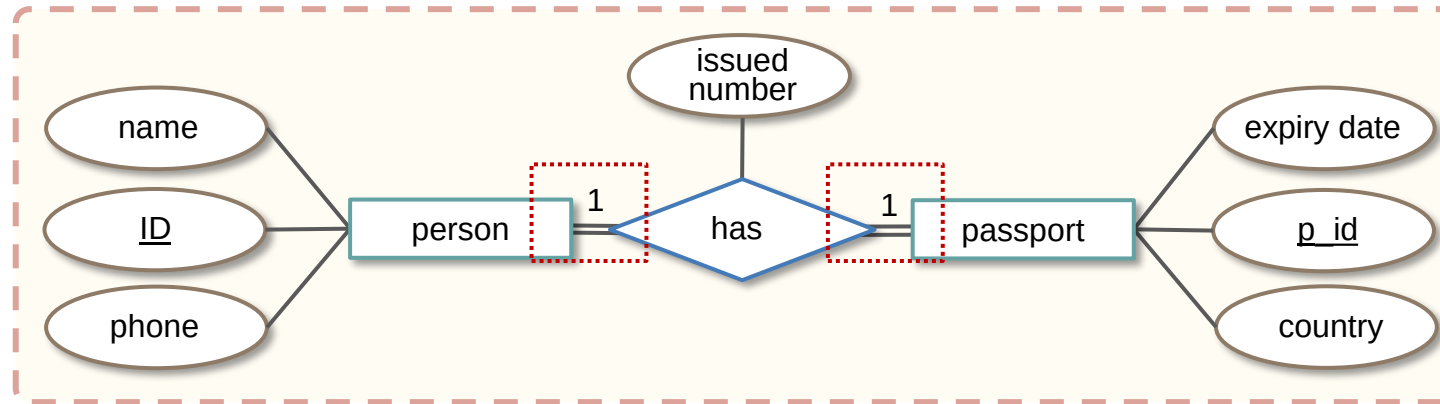
- *person*(ID, name, phone)
- *passport*(p_id, expiry date, country, ID → person, issued number)

Exercise 4 -- Solution



- Create relations for the ER diagram, covering both entity types and relationship types.
 - For one-to-one (1:1) relationships, apply Option 2: Extend either side table.
 - If both sides have total participation, Option 2 also allows merging them into a single relation.

D2.



- ♦ *person*: total participation
- ♦ *passport*: total participation

Extend

- *person*(ID, name, phone)
- *passport*(p_id, expiry date, country)

Merge into a
single relation

- *person*(ID, name, phone, p_id → passport, issued number)
- *passport*(p_id, expiry date, country)

Schema_1 (D2)

Extend

- *person*(ID, name, phone)
- *passport*(p_id, expiry date, country, ID → person, issued number)

Schema_2 (D2)

- *r*(ID, name, phone, p_id, country, expiry date, issued number)
or
- *r*(ID, name, phone, p_id, country, expiry date, issued number)

Schema_3 (D2)

Mapping Relationship Types -- Summary



- Many-to-many (N:M)
 - Create a table for the relationship type.
 - ◆ Add attributes referencing the primary keys of the involved entity type relations
 - ◆ Primary key: set of foreign keys
- One-to-many (1:N) / Many-to-one (N:1)
 - Option 1: Create a separate table for the relationship.
 - Option 2: Extend the “many” side (N-side) table. (*Improved*)
 - ◆ Add foreign key referencing the primary key of the “one” side entity type relation
 - ◆ Add attributes of the relationship type
- One-to-one (1:1)
 - Option 1: Create a separate table for the relationship.
 - Option 2: Extend either side table. (*Improved*)
 - ◆ Add foreign key referencing the primary key of the other entity type relation
 - ◆ Add attributes of the relationship type
 - ◆ It is best to extend the relation of the entity type with total participation.
 - ◆ If both sides have total participation, they can be merged into a single relation; otherwise, keep them separate

Agenda

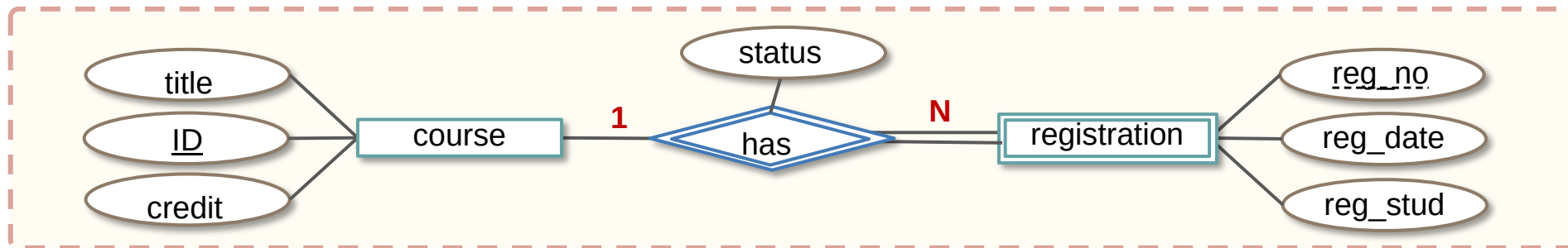


- Review the ER model and relational model
- Mapping basic concepts to relations
 - Entity types
 - Relationship types
- Map complex cases to relations
 - Weak entity types
 - Recursive relationship types
 - N-ary relationship types
 - Special attributes
- Relational modelling of specialization

Mapping Weak Entity Types to Relations



- **Weak entity types** are
 - Existentially dependent on a strong entity type (identifying entity).
 - Uniquely identifiable in combination with the strong entity type's key
- ER diagram notation
 - Weak entity types → double rectangles
 - Discriminator → underlined with a dashed line
 - Identifying relationships (weak entity ↔ owner) → double diamonds

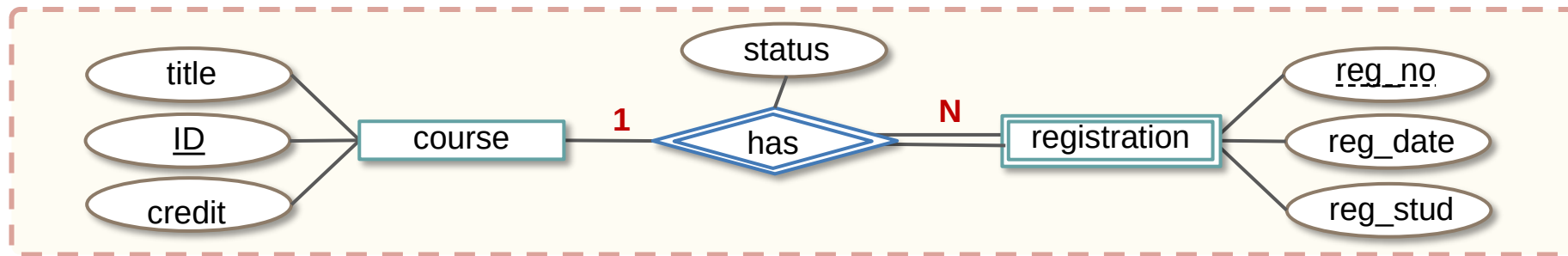


- *Total participation of the weak entity type.*
- *Only in combination with one-to-many (1:N) or many-to-one (N:1) relationship types*
- *The identifying entity type is always on the “1”-side*

Mapping Weak Entity Types to Relations



- Weak entity type: **Create** a new relation of the weak entity type.
 - Add all attributes of the weak entity as **regular attributes**.
 - Add the primary key of the strong entity type (on which it depends) as a **foreign key**.
 - The **primary key of the weak entity type** is :
 - strong entity key + discriminator
- Example

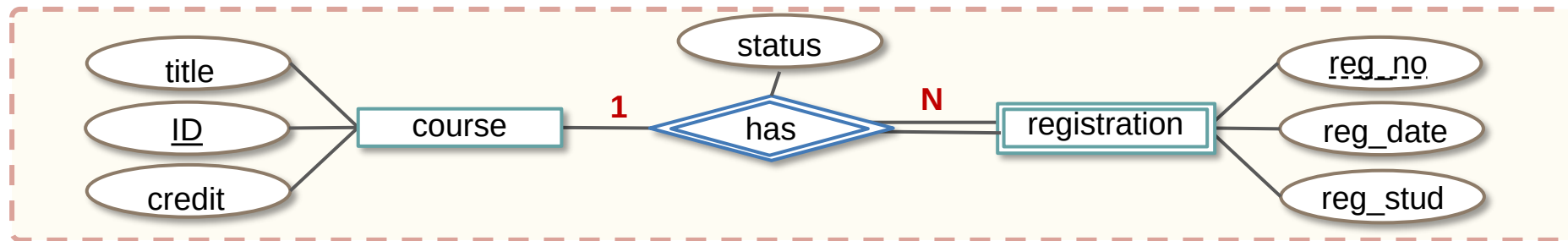


- $course(\underline{ID}, title, credit)$
- $registration(\underline{ID} \rightarrow course, reg_no, reg_date, reg_stud)$

Mapping Weak Entity Types to Relations



- Identifying relationship (One-to-many relationship)
 - Option 1: **Create** a new relation for the identifying relationship.
 - ◆ Add all attributes of the identifying relationship as regular attributes.
 - ◆ Add primary key of the strong entity and the discriminator of the weak entity as foreign keys
 - ◆ Primary key = primary key of the strong entity + discriminator of the weak entity.
- Example

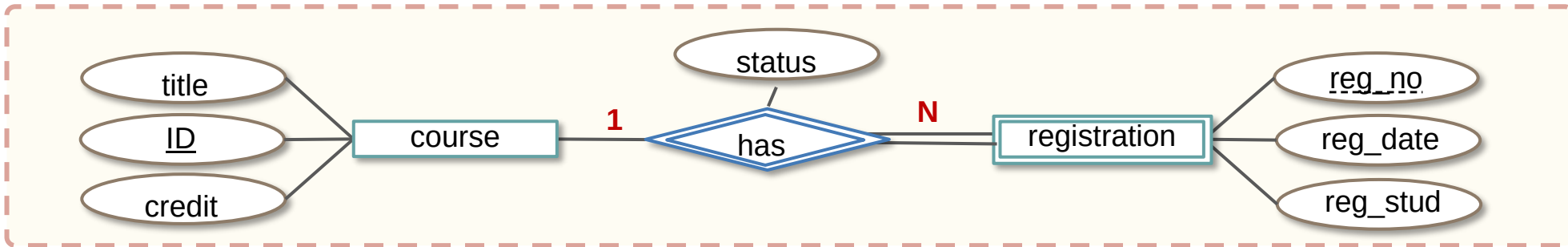


- *course*(ID, title, credit)
- *registration*(ID → *course*, reg_no, reg_date, reg_stud)
- *has*(ID → *course*, reg_no → *registration*, status)

Mapping Weak Entity Types to Relations



- Identifying relationship (One-to-many relationship)
 - Option 2: **Extend** the relation of the **weak entity type**.
 - ◆ The weak entity always has total participation and is on the “many” side.
 - ◆ Add the relationship attribute directly into the weak entity table.
- Example



- $course(\underline{ID}, title, credit)$
- $registration(\underline{ID} \rightarrow course, reg_no, reg_date, reg_stud)$

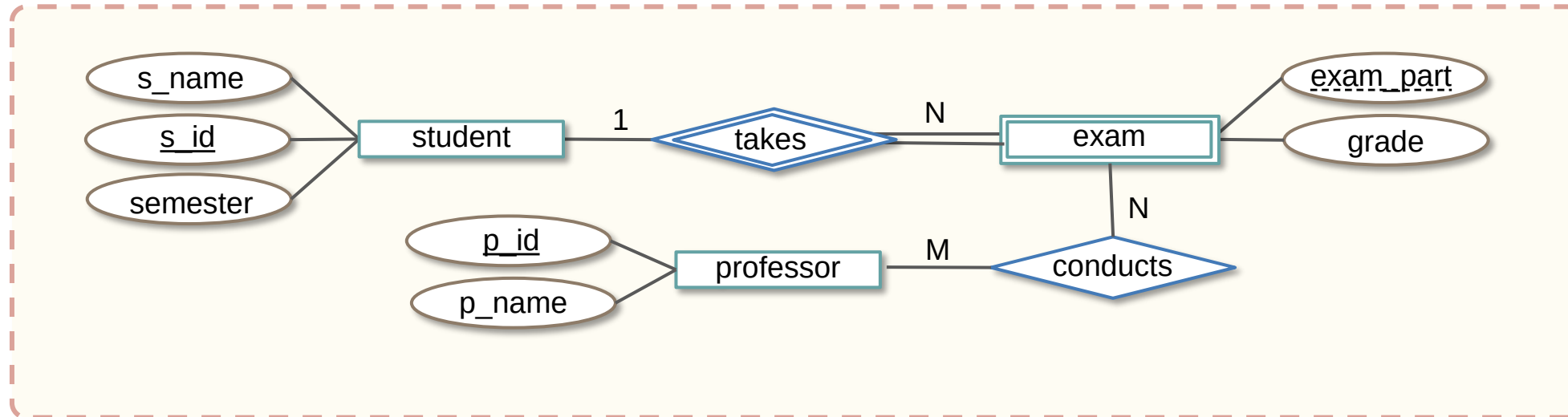
Extend the weak entity type

- $course(\underline{ID}, title, credit)$
- $registration(\underline{ID} \rightarrow course, reg_no, reg_date, reg_stud, status)$

Mapping Weak Entity Types to Relations



- Relationships involving weak entities and other strong entities
 - Example

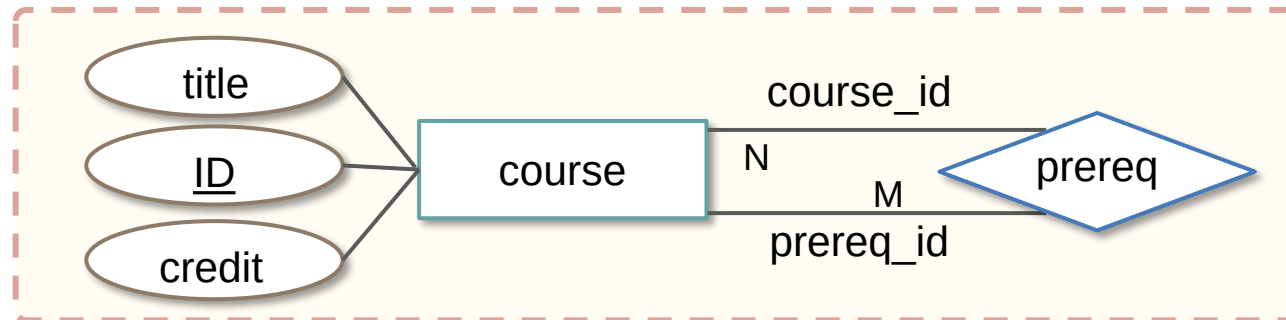


- $exam(\underline{s_id} \rightarrow \text{student}, \underline{exam_part}, grade)$
- $professor(\underline{p_id}, p_name)$
- $conducts(\underline{s_id} \rightarrow \text{exam.s_id}, \underline{exam_part} \rightarrow \text{exam.exam_part}, p_id \rightarrow professor)$

Mapping Recursive relationship types to Relations



- Recursive relationship
 - A relationship where the same entity set participates **more than once**
 - The entity **plays different roles** in the relationship
- If it is a many-to-many (N:M) relationship
 - Map it in the same way as a standard N:M relationship type, but make sure to rename the foreign keys to reflect their different roles.

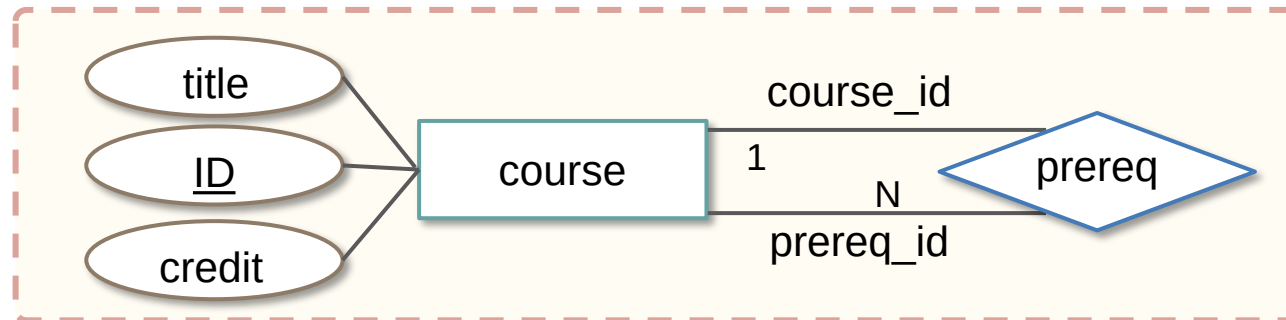


- *course(ID, title, credit)*
- *prereq(course_id → course, prereq_id → course)*

Mapping Recursive relationship types to Relations



- Recursive relationship
 - A relationship where the same entity set participates **more than once**
 - The entity **plays different roles** in the relationship
- If it is a one-to-many (1:N) relationship
 - Map it in the same way as a standard 1:N relationship type, but make sure to rename the foreign keys to reflect their different roles.

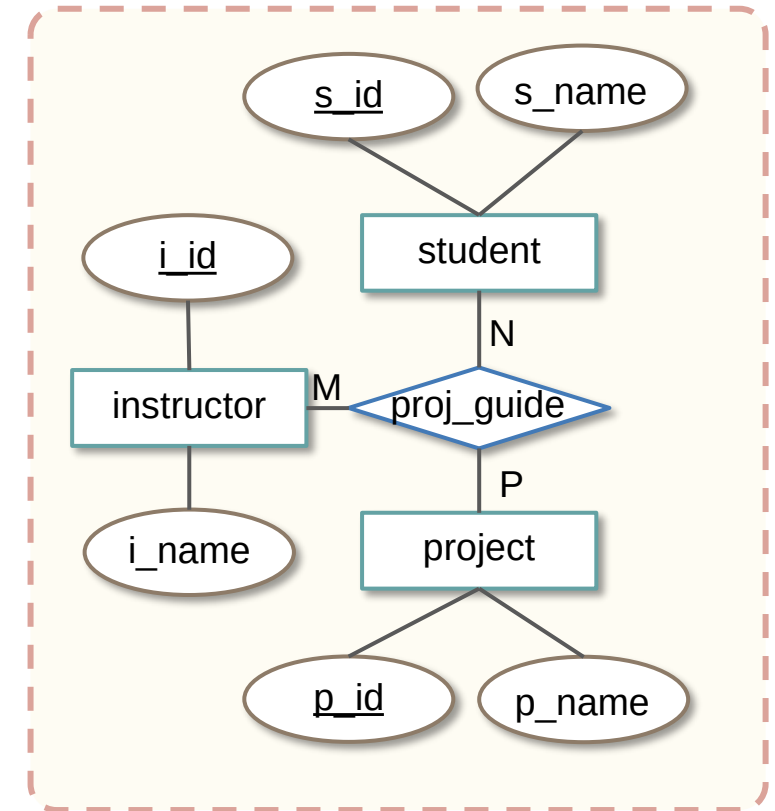


- $course(\underline{ID}, title, credit, prereq_id \rightarrow course)$

Mapping N-ary relationship types to Relations



- Entity types:
 - All participating entity types are mapped according to the standard rules.
 - Example:
 - ◆ *student(s_id, s_name)*
 - ◆ *project(p_id, p_name)*
 - ◆ *instructor(i_id, i_name)*
- N-ary relationship types (N:M:P)
 - Create a new relation that includes the primary keys of all participating entity types as foreign keys. The combination of these foreign keys serves as the primary key of the new relation, along with any attributes of the relationship type.
 - Example:
 - ◆ *proj_guide(s_id → student, p_id → project, i_id → instructor)*



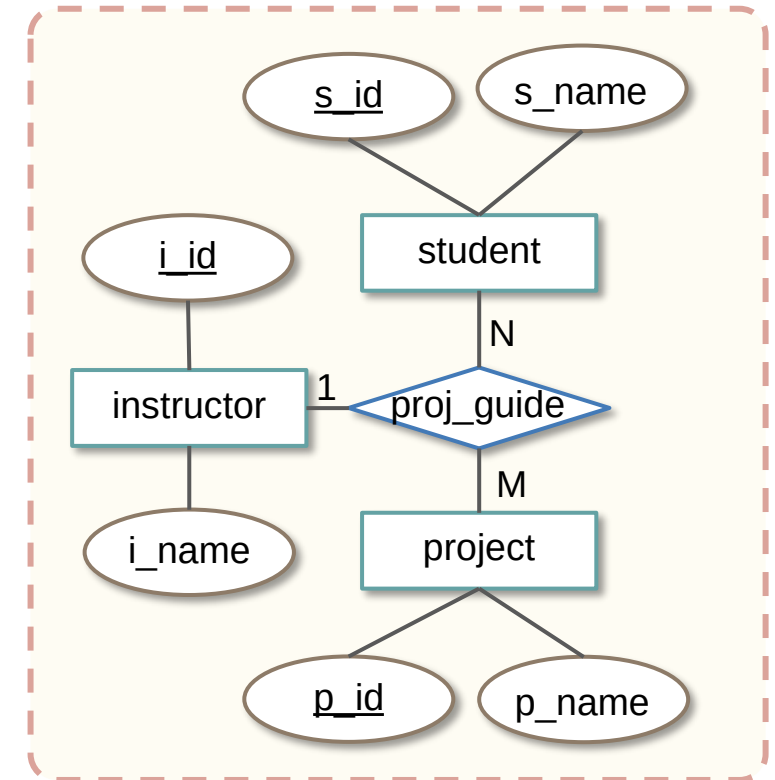
Mapping N-ary relationship types to Relations



- Entity types:
 - All participating entity types are mapped according to the standard rules.
 - Example:
 - ◆ *student(s_id, s_name)*
 - ◆ *project(p_id, p_name)*
 - ◆ *instructor(i_id, i_name)*

- N-ary relationship types (N:M:1)

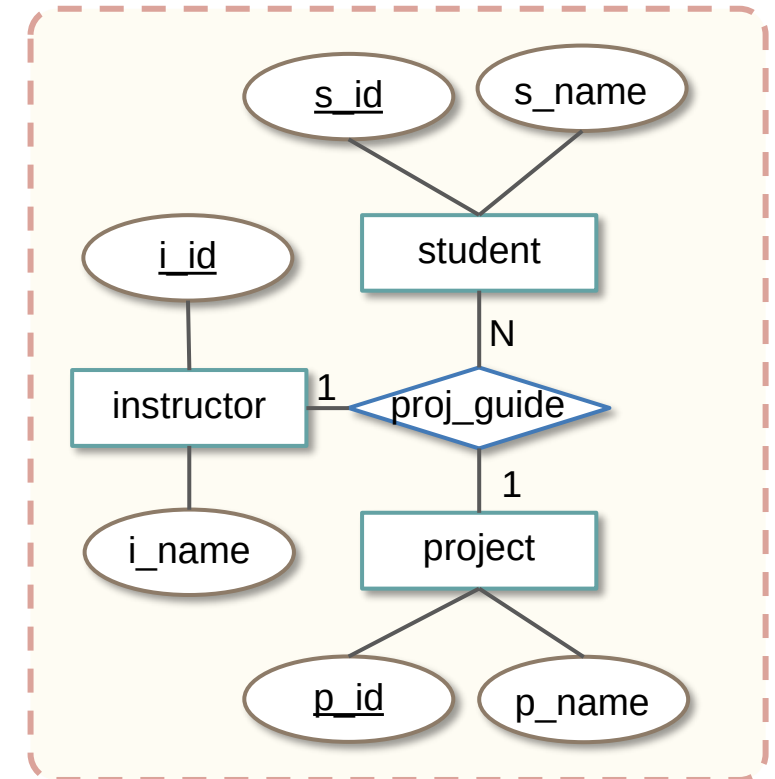
- Create a new relation that includes
 - ◆ The primary keys of the **N-side and M-side entity types** as **foreign keys** and **together as the primary key**.
 - ◆ The primary key of the **1-side entity** type as **a foreign key**.
 - ◆ Any attributes of the relationship type as regular attributes.
- Example:
 - ◆ *proj_guide(s_id → student, p_id → project, i_id → instructor)*



Mapping N-ary relationship types to Relations



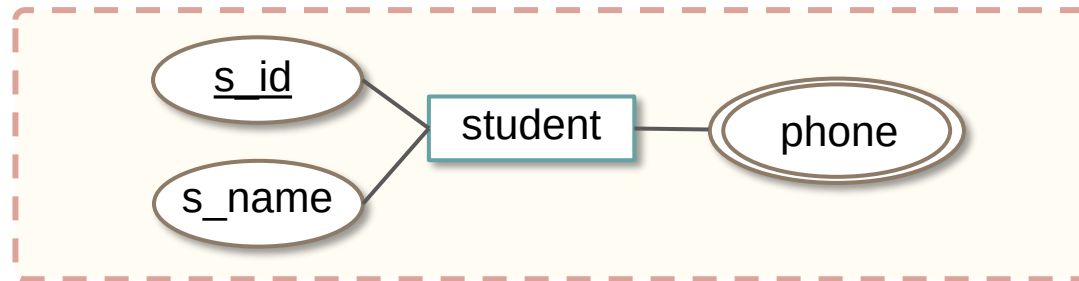
- Entity types:
 - All participating entity types are mapped according to the standard rules.
 - Example:
 - ◆ *student(s_id, s_name)*
 - ◆ *project(p_id, p_name)*
 - ◆ *instructor(i_id, i_name)*
- N-ary relationship types (N:1:1)
 - Create a new relation that includes
 - ◆ The primary key of the N-side entity type as **a foreign key** and **the primary key**.
 - ◆ The primary keys of the two 1-side entity types as **foreign keys**.
 - ◆ Any attributes of the relationship type as regular attributes.
 - Example:
 - ◆ *proj_guide(s_id → student, p_id → project, i_id → instructor)*



Mapping Complex Attributes to Relations



- Multi-valued attributes
 - **Create** a separate relation for each multi-valued attribute
 - ◆ The new relation includes **the primary key of the original entity type (as a foreign key)** and **the multi-valued attribute itself**.
 - ◆ The primary key of this new relation is the combination of **the original entity's primary key and the multi-valued attribute**.
 - Example: A student might have multiple phone numbers



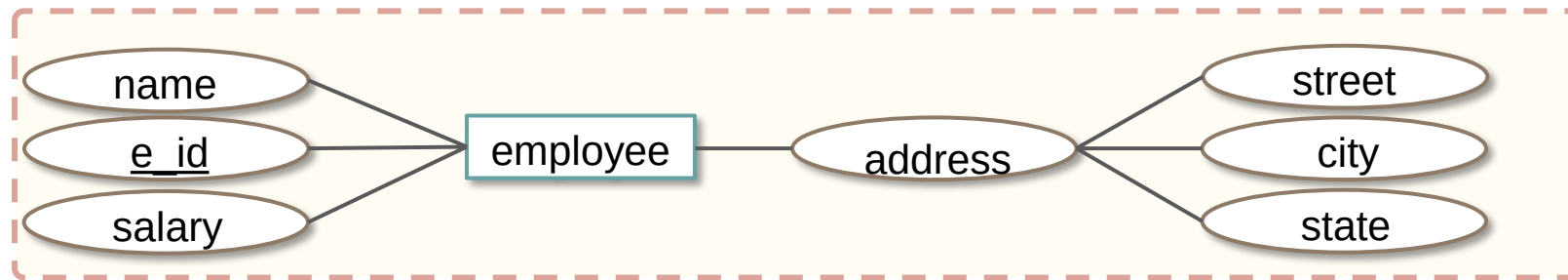
- ◆ *student(s_id, s_name)*
- ◆ *phone(s_id → student, phoneNumber)*

Mapping Complex Attributes to Relations



- Composite attributes

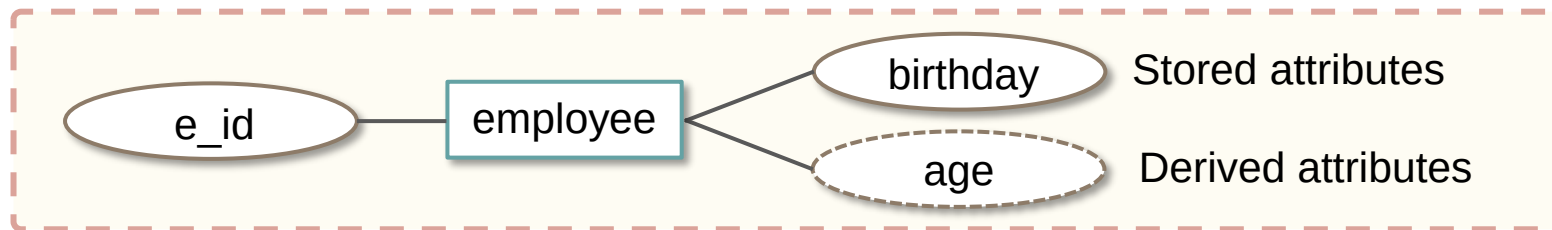
- **Include** the component attributes in the relation
- Example: An address can be modelled as a string or composed of street, city and state



♦ *employee(e_id, name, salary, state, city, street)*

- Derived attributes

- Ignored during mapping to relations, can be added later by using views.
- Example: Age can be derived from a stored birth-date



♦ *employee(e_id, birthday)*

Overview of the Steps



S1. Regular entity type

- Create a relation, consider **special attribute types**

S2. Weak entity type

- Create a relation

S3. 1:1 binary relationship type

- Extend a relation with foreign key

S4. 1:N binary relationship type

- Extend a relation with foreign key

S5. N:M relationship type

- Create a relation

S6. N-ary relationship type

- Create a relation

S7. Recursive relationship type

- Create a relation

Agenda

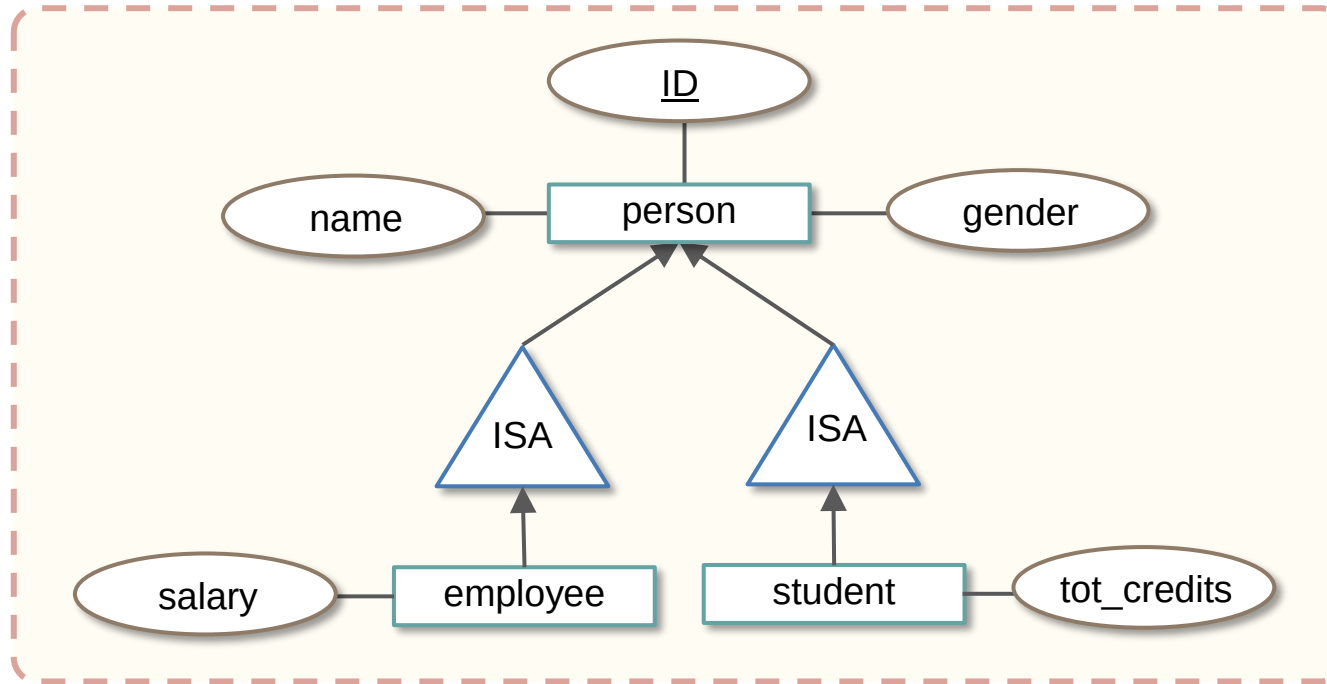


- Review the ER model and relational model
- Mapping basic concepts to relations
 - Entity types
 - Relationship types
- Map complex cases to relations
 - Weak entity types
 - Recursive relationship types
 - N-ary relationship types
 - Special attributes
- Relational modelling of specialization

Relational Modelling of Specialization



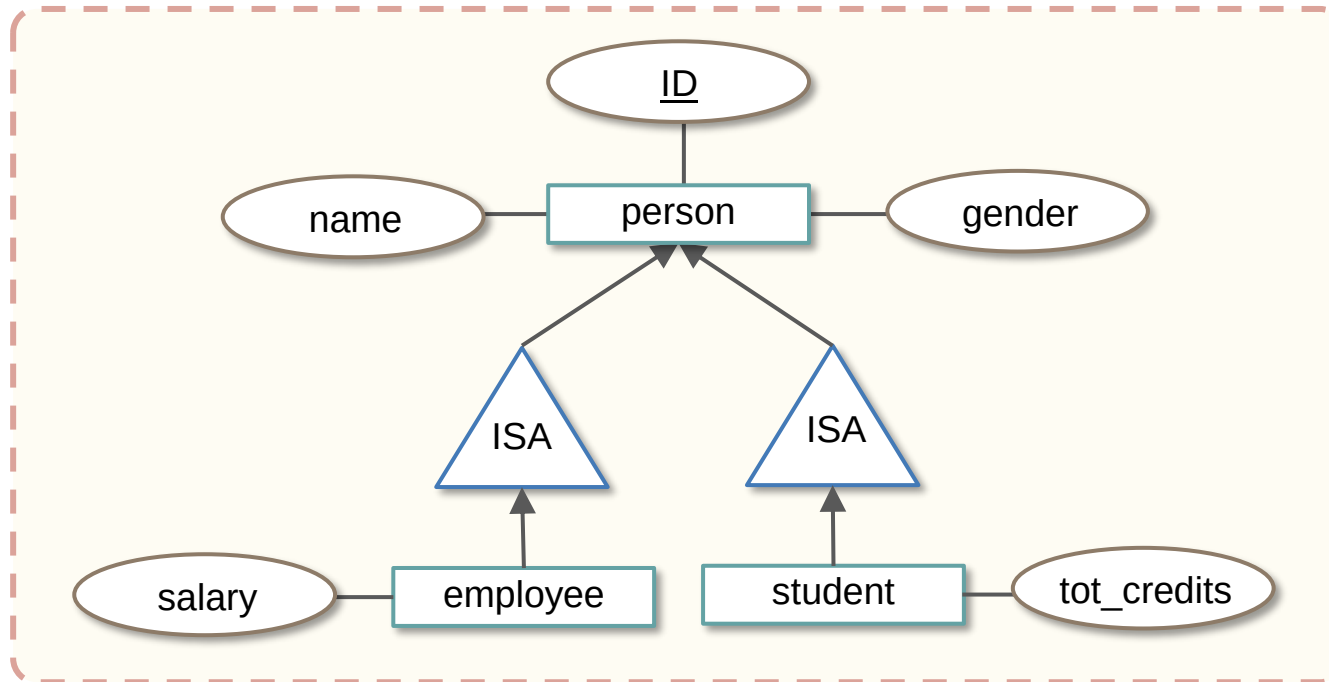
- The relational model does not support specialization and cannot express inheritance.
- Specialization is simulated.



Relational Modelling of Specialization



- Alternative 1: main classes
 - A particular entity is mapped to a single tuple in a single relation (to its main class).



person

<u>ID</u>	name	gender
12930	Sophia	F
18231	Olivia	F
72201	Ethan	M

employee

<u>ID</u>	name	gender	salary
12121	Wu	M	90000
76766	Crick	M	72000
83821	Brandt	F	92000

student

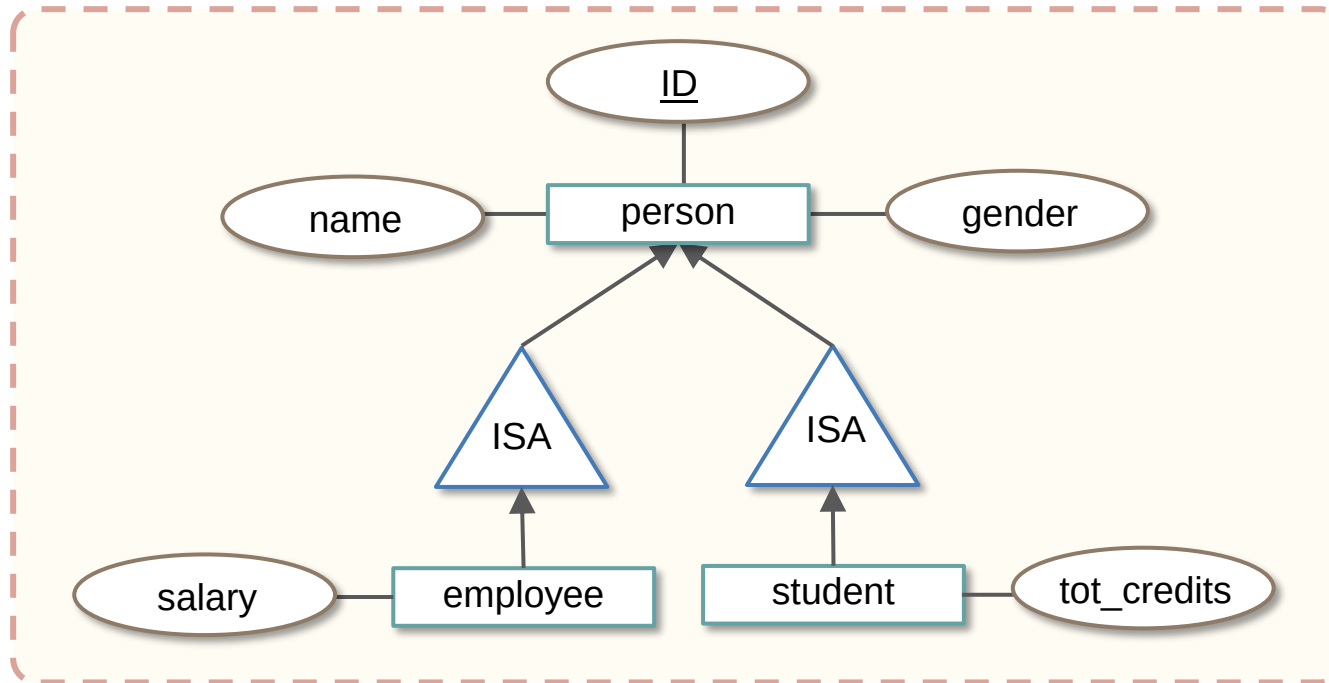
<u>ID</u>	name	gender	tot_credits
10001	Alice	F	100
10002	Anna	F	110
10006	David	M	125

- ◆ *person*(ID, name, gender)
- ◆ *employee*(ID, name, gender, salary)
- ◆ *student*(ID, name, gender, tot_credits)

Relational Modelling of Specialization



- Alternative 2: partitioning
 - Parts of a particular entity are mapped to multiple relations, the key is duplicated.



- ◆ $person(\underline{ID}, name, gender)$
- ◆ $employee(\underline{ID} \rightarrow \underline{person}, salary)$
- ◆ $student(\underline{ID} \rightarrow \underline{person}, tot_credits)$

person

<u>ID</u>	name	gender
10001	Alice	F
10002	Anna	F
10006	David	M
12121	Wu	M
12930	Sophia	F
18231	Olivia	F
72201	Ethan	M
76766	Crick	M
83821	Brandt	F

employee

<u>ID</u>	salary
12121	90000
76766	72000
83821	92000

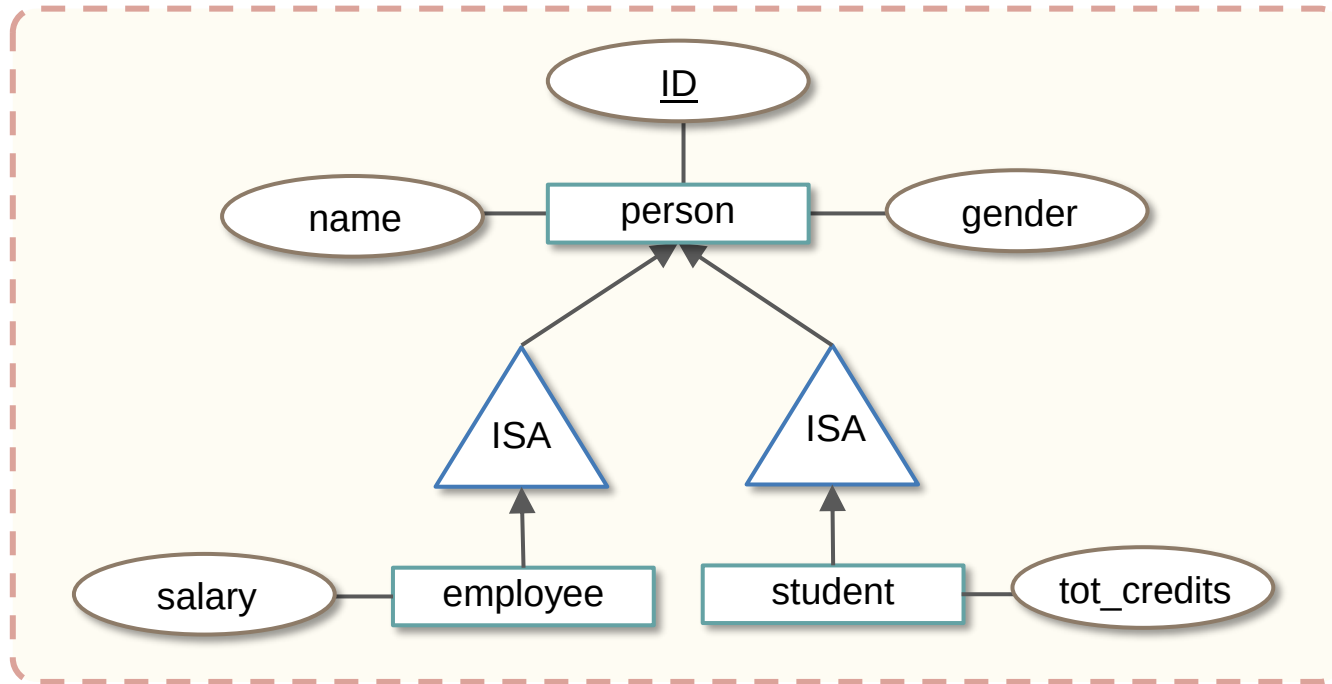
student

<u>ID</u>	tot_credits
10001	100
10002	110
10006	125

Relational Modelling of Specialization



- Alternative 3: full redundancy
 - A particular entity is stored redundantly in the relations with all its inherited attributes.



- ◆ *person*(ID, name, gender)
- ◆ *employee*(ID, name, gender, salary)
- ◆ *student*(ID, name, gender, tot_credits)

person

<u>ID</u>	name	gender
10001	Alice	F
10002	Anna	F
10006	David	M
12121	Wu	M
12930	Sophia	F
18231	Olivia	F
72201	Ethan	M
76766	Crick	M
83821	Brandt	F

employee

<u>ID</u>	name	gender	salary
12121	Wu	M	90000
76766	Crick	M	72000
83821	Brandt	F	92000

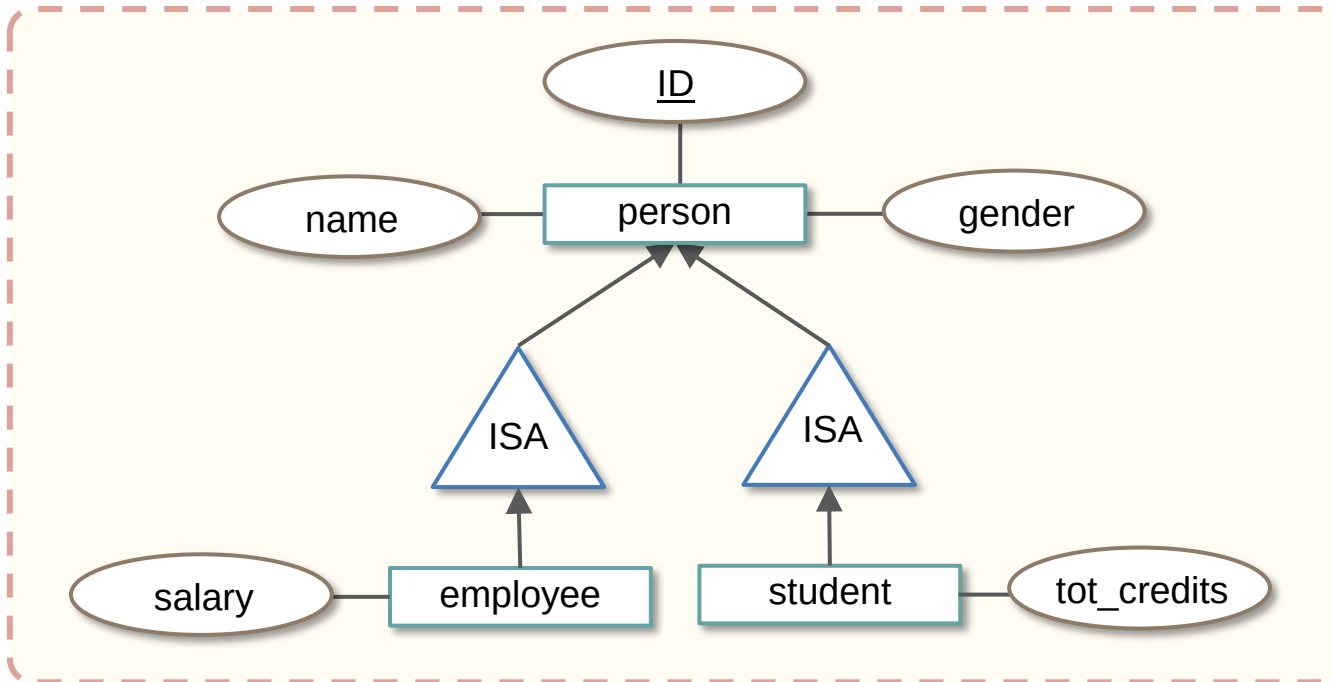
student

<u>ID</u>	name	gender	tot_credits
10001	Alice	F	100
10002	Anna	F	110
10006	David	M	125

Relational Modelling of Specialization



- Alternative 4: single relation
 - All entities are stored in a single relation. An additional attribute encodes the membership in a particular entity type.



◆ *person*(ID, name, gender, type, salary, tot_credits)

person

<u>ID</u>	name	gender	type	salary	credits
10001	Alice	F	student	null	100
10002	Anna	F	student	null	110
10006	David	M	student	null	125
12121	Wu	M	employee	90000	null
12930	Sophia	F	person	null	null
18231	Olivia	F	person	null	null
72201	Ethan	M	person	null	null
76766	Crick	M	employee	72000	null
83821	Brandt	F	employee	92000	null

Summary



- Review the ER model and relational model
- Mapping basic concepts to relations
 - Entity types
 - Relationship types
- Map complex cases to relations
 - Weak entity types
 - Recursive relationship types
 - N-ary relationship types
 - Special attributes
- Relational modelling of specialization



Any Questions?

Lecture 9

The Entity Relationship Model (Part 2)

Yumeng Song

yumengs@cs.aau.dk

Center for Data-Intensive Systems

Reference



- Abraham et al.. Database System Concepts (seventh edition) – Chapter 6
- Tiantian Liu. Database Systems(SW5) - 6. ER Model (Part 2)
- Christian S. Jensen. Database Systems - The Entity Relationship Model